BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM

ĐỒ ÁN CƠ SỞ

HỆ THỐNG LẬP KẾ HOẠCH DU LỊCH THÔNG MINH

Ngành: **CÔNG NGHỆ THÔNG TIN**

Chuyên ngành: **CÔNG NGHỆ PHẦN MỀM**

Giảng viên hướng dẫn : Trần Thị Vân Anh

Sinh viên thực hiện :

Họ tên:	MSSV:	Lớp:
Nguyễn Xuân Phú	2380601708	23DTHC4
Nguyễn Hữu Dũng	2380600299	23DTHC4

TP. Hồ Chí Minh, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM

ĐỒ ÁN CƠ SỞ

HỆ THỐNG LẬP KẾ HOẠCH DU LỊCH THÔNG MINH

Ngành: **CÔNG NGHỆ THÔNG TIN**

Chuyên ngành: **CÔNG NGHỆ PHẦN MỀM**

Giảng viên hướng dẫn : Trần Thị Vân Anh

Sinh viên thực hiện :

Họ tên:	MSSV:	Lớp:
Nguyễn Xuân Phú	2380601708	23DTHC4
Nguyễn Hữu Dũng	2380600299	23DTHC4

TP. Hồ Chí Minh, 2026

LỜI MỞ ĐẦU

Việc lập kế hoạch du lịch thường yêu cầu mọi người phải tìm kiếm thông tin về địa điểm, sắp xếp lịch trình, gây tốn nhiều thời gian và công sức.

Xuất phát từ nhu cầu đó, nhóm chúng em thực hiện dự án "Hệ thống lập kế hoạch du lịch thông minh" nhằm xây dựng một ứng dụng web hỗ trợ người dùng tạo lịch trình du lịch tự động dựa trên điểm đến, số ngày, ngân sách và sở thích cá nhân. Hệ thống sử dụng Gemini AI để đề xuất kế hoạch phù hợp.

Dự án được phát triển bằng các công nghệ hiện đại như ReactJS, ExpressJS, Prisma ORM, MySQL, JWT Authentication và OpenStreetMap. Trong quá trình thực hiện, nhóm đã vận dụng kiến thức về phân tích thiết kế hệ thống, lập trình web, cơ sở dữ liệu và tích hợp API.

Qua dự án này, nhóm mong rằng có thể tạo ra 1 công cụ giúp mọi người có thể lên kế hoạch 1 cách phù hợp và nhanh chóng trong cuộc sống vội vã hiện nay.

Nhóm xin chân thành cảm ơn cô Trần Thị Vân Anh đã hỗ trợ để nhóm hoàn thành đồ án.

LỜI CAM ĐOAN

Chúng em xin cam đoan đồ án này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên phụ trách.

Toàn bộ nội dùng trong báo cáo được thực hiện dựa trên kiến thức đã học, quá trình tìm hiểu tài liệu, đưa ra ý tưởng và tự xây dựng chương trình. Các thông tin, tài liệu tham khảo được sử dụng trong báo cáo đều được ghi rõ nguồn gốc.

Chúng em xin chịu hoàn toàn trách nhiệm về tính trung thực của nội dùng báo cáo và sản phẩm thực hiện trong đồ án này.

Sinh viên thực hiện:

MSSV: 2380601708 - Họ và tên: Nguyễn Xuân Phú

MSSV: 2380600299 - Họ và tên: Nguyễn Hữu Dũng

MỤC LỤC

LỜI MỞ ĐẦU .....

LỜI CAM ĐOAN .....

MỤC LỤC 1

DANH MỤC VIẾT TẮT 3

DANH MỤC HÌNH ẢNH 4

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 5

1.1Giới thiệu đề tài 5

1.2 Các nghiên cứu và hệ thống liên quan 5

1.3 Nhiệm vụ đồ án 6

1.3.1 Tính cấp thiết và lý do chọn đề tài 6

1.3.2 Ý nghĩa khoa học 7

1.3.3 Ý nghĩa thực tiễn 7

1.3.4 Mục tiêu nghiên cứu 7

1.3.5 Đối tượng nghiên cứu 8

1.3.6 Phạm vi nghiên cứu 9

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 10

2.1 Tổng quan về hệ thống lập kế hoạch du lịch 10

2.2 Trí tuệ nhân tạo và Prompt Engineering 10

2.3 Vector Database và Semantic Search 12

2.4 OpenStreetMap và Nominatim API 13

2.5 Kiến trúc Client – Server 13

2.6 ReactJS 14

2.7 Vite 15

2.8 ExpressJS 16

2.9 Node.js 17

2.10 Python FastAPI 17

2.11 Prisma ORM 18

2.12 MySQL **19**
2.13 Axios **20**
2.14 LeafletJS **20**
CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM **22**
3.1 Phân tích yêu cầu hệ thống **22**
3.1.1 Yêu cầu chức năng 22
3.1.2 Yêu cầu phi chức năng 22
3.1.3 Các tác nhân hệ thống 22
3.2 Phân tích và thiết kế hệ thống **23**
3.2.1 Sơ đồ Use Case tổng quát 23
3.2.2 Sơ đồ Activity Diagram 23
3.2.3 Thiết kế cơ sở dữ liệu (ERD) 25
3.2.4 Thiết kế kiến trúc hệ thống 25
3.3 Xây dựng và triển khai hệ thống **26**
3.3.1 Xây dựng frontend 26
3.3.2 Xây dựng backend 30
3.3.3 Tích hợp trí tuệ nhân tạo 32
3.3.4 Triển khai hệ thống 33
CHƯƠNG 4. KẾT LUẬN VÀ KIẾN NGHỊ **35**
4.1 Kết luận **35**
4.2 Hạn chế của hệ thống **35**
4.3 Hướng phát triển **36**
4.4 Kiến nghị **36**
TÀI LIỆU THAM KHẢO **37**

DANH MỤC VIẾT TẮT

Từ viết tắt	Ý nghĩa
AI	Artificial Intelligence
API	Application Programming Interface
ORM	Object Relational Mapping
UI	User Interface
UX	User Experience
JWT	JSON Web Token
REST API	Representational State Transfer API
OSM	OpenStreetMap
CRUD	Create - Read - Update - Delete

DANH MỤC HÌNH ẢNH

Hình 2.1: Quy trình Prompt Engineering và tối ưu hóa phản hồi AI 11

Hình 2.2: Quy trình tìm kiếm ngữ nghĩa bằng Vector Embedding 12

Hình 2.3: Giao diện tìm kiếm địa điểm bằng Nominatim API 13

Hình 2.4: Kiến trúc Client – Server của hệ thống 14

Hình 2.5: Các thành phần chính trong ReactJS và cơ chế Virtual DOM 15

Hình 2.6: So sánh tốc độ tải module trước và sau khi sử dụng Vite 16

Hình 2.7:Kiến trúc xử lý request và response trong ExpressJS 17

Hình 2.8: Mô hình Event Loop và Non-blocking I/O trong Node.js 17

Hình 2.9: Quy trình truy vấn cơ sở dữ liệu bằng Prisma ORM 18

Hình 2.10: Mô hình kết nối và xử lý dữ liệu với MySQL Database 19

Hình 2.11: Mô hình giao tiếp API giữa Frontend và Backend bằng Axios 20

Hình 2.12: LeafletJS tích hợp bản đồ tương tác trong ứng dụng web 21

Hình 3.1: USE CASE DIAGRAM hệ thống 23

Hình 3.2: Activity - User 24

Hình 3.3: Activity - Admin 24

Hình 3.4: Giao diện đăng nhập hệ thống 26

Hình 3.5: Giao diện bảng điều khiển chính (Dashboard) 27

Hình 3.6: Giao diện nhập thông tin cho AI Planner 28

Hình 3.7: Giao diện lập kế hoạch thủ công với bản đồ tương tác 29

Hình 3.8: Giao diện khám phá và gợi ý địa điểm du lịch 29

Hình 3.9: Giao diện Admin Dashboard 31

Hình 3.10: Giao diện quản lý người dùng 31

Hình 3.11:Giao diện quản lý khám phá địa điểm 32

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1Giới thiệu đề tài

Đề tài "Hệ thống lập kế hoạch du lịch thông minh" được xây dựng nhằm hỗ trợ người dùng tạo lịch trình du lịch tối ưu dựa trên nhu cầu thực tế. Hệ thống ứng dụng Gemini AI kết hợp Prompt Engineering để sinh lịch trình tự động. Đồng thời, hệ thống còn hỗ trợ tìm kiếm địa điểm bằng OpenStreetMap API và quản lý toàn bộ chuyến đi.

Việc ứng dụng AI trong lĩnh vực du lịch giúp tăng trải nghiệm người dùng, giảm thời gian lập kế hoạch và mang lại sự tiện lợi trong quá trình tổ chức chuyến đi.

1.2 Các nghiên cứu và hệ thống liên quan

Trong những năm gần đây, nhiều hệ thống hỗ trợ lập kế hoạch du lịch thông minh đã được phát triển nhằm tối ưu trải nghiệm người dùng. Các nền tảng phổ biến như TripAdvisor, Booking.com hay Expedia cho phép tìm kiếm địa điểm, khách sạn và xây dựng lịch trình cơ bản dựa trên đánh giá cộng đồng.

Bên cạnh các hệ thống thương mại, nhiều nghiên cứu học thuật cũng tập trung vào bài toán gợi ý du lịch cá nhân hóa. Nghiên cứu "Efficient itinerary recommendation via personalized POI selection and pruning"[1] đề xuất phương pháp lựa chọn địa điểm tham quan dựa trên sở thích và thời gian của người dùng nhằm tối ưu lịch trình du lịch. Ngoài ra, nghiên cứu "Your trip, your way: An adaptive tourism recommendation system"[2] cho thấy trí tuệ nhân tạo và hệ gợi ý thích ứng có khả năng cải thiện mức độ cá nhân hóa trong quá trình lập kế hoạch chuyến đi.

Hiện nay, nhiều hệ thống đã bắt đầu ứng dụng AI tạo sinh (Generative AI) để xây dựng lịch trình tự động thông qua ngôn ngữ tự nhiên. Người dùng chỉ cần nhập nhu cầu cơ bản như địa điểm, ngân sách hoặc sở thích, hệ thống sẽ tự động đề xuất kế hoạch phù hợp. Tuy nhiên, các hệ thống hiện tại vẫn tồn tại một số hạn chế như phụ thuộc dữ liệu bên ngoài, kết quả đôi khi chưa chính xác hoặc chưa phù hợp hoàn toàn với điều kiện thực tế của người dùng.

Từ các nghiên cứu và hệ thống đã khảo sát, đề tài xây dựng một hệ thống lập kế hoạch du lịch thông minh trên nền tảng web, kết hợp Gemini AI, OpenStreetMap API

và tìm kiếm ngữ nghĩa nhằm hỗ trợ người dùng tạo lịch trình nhanh chóng và thuận tiện hơn.

1.3 Nhiệm vụ đồ án

1.3.1 Tính cấp thiết và lý do chọn đề tài

Hiện nay, nhu cầu du lịch cá nhân ngày càng gia tăng, nhiều người có xu hướng tự tổ chức chuyến đi để chủ động về thời gian, chi phí và trải nghiệm. Tuy nhiên, việc xây dựng một kế hoạch du lịch hoàn chỉnh thường đòi hỏi người dùng phải tìm kiếm và tổng hợp rất nhiều thông tin như địa điểm tham quan, phương tiện di chuyển, thời gian phù hợp, chi phí dự kiến và các hoạt động theo sở thích cá nhân. Quá trình này không chỉ mất nhiều thời gian mà còn dễ dẫn đến việc lựa chọn thiếu tối ưu, khiến chuyến đi kém hiệu quả.

Bên cạnh đó, sự phát triển mạnh mẽ của trí tuệ nhân tạo đã mở ra nhiều cơ hội ứng dụng trong việc tự động hóa các tác vụ đòi hỏi khả năng phân tích và đề xuất. Thực tế, AI đã có thể hỗ trợ con người đánh giá nhu cầu của người dùng, lựa chọn địa điểm phù hợp và xây dựng lịch trình hợp lý dựa trên các tiêu chí như ngân sách, số ngày và sở thích cá nhân. Điều này góp phần nâng cao trải nghiệm người dùng và giảm đáng kể công sức trong quá trình chuẩn bị chuyến đi.

Từ góc độ học thuật, đề tài này là cơ hội để vận dụng tổng hợp nhiều kiến thức đã được học như phân tích và thiết kế hệ thống, phát triển ứng dụng web, xây dựng cơ sở dữ liệu và ứng dụng trí tuệ nhân tạo. Việc kết hợp các công nghệ hiện đại như ReactJS, ExpressJS, MySQL, Prisma ORM, OpenStreetMap API, Vector Database và Gemini AI giúp nhóm tiếp cận với quy trình phát triển phần mềm và nâng cao kỹ năng triển khai hệ thống hoàn chỉnh.

Xuất phát từ những lý do trên, nhóm quyết định lựa chọn đề tài “Hệ thống lập kế hoạch du lịch thông minh” nhằm xây dựng một ứng dụng web hỗ trợ người dùng tạo lịch trình du lịch bằng trí tuệ nhân tạo, đồng thời đáp ứng mục tiêu học tập, nghiên cứu và phát triển một sản phẩm có giá trị thực tiễn.

1.3.2 Ý nghĩa khoa học

Đề tài "Hệ thống lập kế hoạch du lịch thông minh" có ý nghĩa khoa học trong việc nghiên cứu và ứng dụng các phương pháp, công nghệ hiện đại để giải quyết bài toán lập kế hoạch du lịch tự động. Đề tài kết hợp kiến thức thuộc nhiều lĩnh vực như phân tích và thiết kế hệ thống, phát triển ứng dụng web, quản trị cơ sở dữ liệu, trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên.

Một trong những đóng góp chính của đề tài là việc ứng dụng Gemini AI kết hợp với Prompt Engineering để tạo lịch trình du lịch dựa trên các thông tin đầu vào như địa điểm, số ngày, ngân sách và sở thích của người dùng. Cách tiếp cận này cho thấy khả năng khai thác mô hình ngôn ngữ lớn trong việc hỗ trợ ra quyết định và cá nhân hóa kết quả cho từng nhu cầu cụ thể.

1.3.3 Ý nghĩa thực tiễn

Đề tài "Hệ thống lập kế hoạch du lịch thông minh" mang lại giá trị thực tiễn rõ rệt trong việc hỗ trợ người dùng chuẩn bị cho các chuyến đi một cách thuận tiện và hiệu quả hơn. Thay vì phải tự tìm kiếm thông tin về địa điểm, lên lịch trình và tính toán chi phí, người dùng chỉ cần nhập một số thông tin cơ bản như điểm đến, thời gian, ngân sách và sở thích cá nhân. Hệ thống sẽ tự động đề xuất kế hoạch phù hợp, giúp rút ngắn đáng kể thời gian chuẩn bị.

Ngoài ý nghĩa đối với người dùng, đề tài còn cho thấy khả năng ứng dụng của các công nghệ hiện đại như ReactJS, ExpressJS, MySQL và Gemini AI vào một bài toán thực tế. Kết quả của đề tài có thể được tiếp tục phát triển thành một sản phẩm hoàn chỉnh, đáp ứng nhu cầu lập kế hoạch du lịch cá nhân hoặc hỗ trợ cho các doanh nghiệp hoạt động trong lĩnh vực du lịch.

1.3.4 Mục tiêu nghiên cứu

Mục tiêu tổng quát của dự án là xây dựng một hệ thống lập kế hoạch du lịch thông minh trên nền tảng web, hỗ trợ người dùng tạo lịch trình du lịch tự động bằng trí tuệ nhân tạo dựa trên các tiêu chí như điểm đến, số ngày, ngân sách và sở thích cá nhân.

Các mục tiêu cụ thể của dự án bao gồm:

- Phân tích yêu cầu và thiết kế kiến trúc hệ thống theo mô hình client-server.
- Xây dựng giao diện web thân thiện bằng ReactJS và Vite.
- Phát triển RESTful API bằng ExpressJS và Node.js.
- Thiết kế và triển khai cơ sở dữ liệu MySQL sử dụng Prisma ORM.
- Tích hợp Gemini AI để sinh lịch trình du lịch tự động.
- Ứng dụng Prompt Engineering để tối ưu chất lượng kết quả từ AI.
- Xây dựng Vector Database phục vụ tìm kiếm ngữ nghĩa và gợi ý địa điểm.
- Tích hợp OpenStreetMap API để tìm kiếm địa điểm và hiển thị bản đồ.
- Phát triển chức năng lập kế hoạch thủ công và quản lý chuyến đi.
- Kiểm thử và đánh giá hiệu năng, tính ổn định của hệ thống.

1.3.5 Đối tượng nghiên cứu

Đối tượng nghiên cứu của dự án là các phương pháp, công nghệ và mô hình xây dựng hệ thống hỗ trợ lập kế hoạch du lịch thông minh trên nền tảng web. Trọng tâm nghiên cứu tập trung vào việc ứng dụng trí tuệ nhân tạo tạo sinh (Generative AI) để phân tích nhu cầu người dùng và tự động đề xuất lịch trình du lịch phù hợp dựa trên các thông tin đầu vào như điểm đến, số ngày, ngân sách và sở thích cá nhân.

Bên cạnh đó, đề tài nghiên cứu các kỹ thuật tìm kiếm và gợi ý địa điểm du lịch thông qua dữ liệu bản đồ số, trong đó sử dụng OpenStreetMap như một nguồn dữ liệu địa lý mở được cộng đồng toàn cầu đóng góp.

Ngoài ra, đề tài còn nghiên cứu kiến trúc hệ thống web theo mô hình client–server, các kỹ thuật xác thực người dùng bằng JWT, thiết kế RESTful API, quản lý dữ liệu bằng MySQL và Prisma ORM, cũng như ứng dụng Vector Database để hỗ trợ tìm kiếm ngữ nghĩa. Những nội dung này là cơ sở để xây dựng một hệ thống có khả năng tự động hóa quá trình lập kế hoạch du lịch và hỗ trợ người dùng quản lý toàn bộ thông tin chuyến đi một cách thuận tiện và hiệu quả.

1.3.6 Phạm vi nghiên cứu

Dự án tập trung nghiên cứu và xây dựng một hệ thống hỗ trợ lập kế hoạch du lịch thông minh trên nền tảng web, hướng đến đối tượng người dùng cá nhân có nhu cầu tự tổ chức các chuyến đi. Hệ thống được phát triển theo mô hình client–server, bao gồm giao diện người dùng xây dựng bằng ReactJS, hệ thống backend sử dụng ExpressJS và dịch vụ trí tuệ nhân tạo được triển khai bằng Python FastAPI.

Về mặt chức năng, phạm vi nghiên cứu bao gồm các nội dung chính như tạo lịch trình du lịch tự động bằng Gemini AI, lập kế hoạch thủ công trên bản đồ, tìm kiếm và khám phá địa điểm, quản lý danh sách chuyến đi và xây dựng trang quản trị dữ liệu. Hệ thống cho phép người dùng nhập các thông tin như điểm đến, số ngày, ngân sách và sở thích cá nhân để nhận được lịch trình được đề xuất tự động.

Đề tài sử dụng OpenStreetMap làm nguồn dữ liệu bản đồ và địa điểm. Đây là dự án bản đồ mở được cộng đồng trên toàn thế giới xây dựng và cập nhật liên tục, cung cấp dữ liệu địa lý miễn phí cho website và ứng dụng. Ngoài ra, hệ thống ứng dụng Vector Database để hỗ trợ tìm kiếm ngữ nghĩa và xếp hạng địa điểm theo mức độ phù hợp với nhu cầu của người dùng.

Về phạm vi dữ liệu, đề tài ưu tiên các địa điểm du lịch trong lãnh thổ Việt Nam nhằm bảo đảm tính thực tiễn và thuận tiện trong quá trình kiểm thử. Hệ thống chưa tập trung vào các chức năng thương mại như đặt vé máy bay, đặt khách sạn, thanh toán trực tuyến, tích hợp mạng xã hội hoặc phát triển ứng dụng di động. Những nội dung này nằm ngoài giới hạn nghiên cứu của đồ án và được xem là hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về hệ thống lập kế hoạch du lịch

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và trí tuệ nhân tạo, các hệ thống thông minh ngày càng được quan tâm và ứng dụng rộng rãi. Thay vì phải tự tìm kiếm thông tin từ nhiều nguồn khác nhau, người dùng hiện nay có xu hướng sử dụng các nền tảng hỗ trợ tự động để tiết kiệm thời gian và nâng cao trải nghiệm du lịch.

Hệ thống lập kế hoạch du lịch thông minh là một loại ứng dụng có khả năng hỗ trợ người dùng xây dựng lịch trình dựa trên nhiều tiêu chí như địa điểm, ngân sách, thời gian và sở thích cá nhân. Các hệ thống hiện đại thường tích hợp bản đồ số, công nghệ tìm kiếm ngữ nghĩa và trí tuệ nhân tạo nhằm tối ưu khả năng gợi ý địa điểm và sắp xếp lịch trình hợp lý.

Trong đề tài này, hệ thống được xây dựng theo hướng kết hợp giữa ứng dụng web và AI tạo sinh. Người dùng chỉ cần nhập các thông tin cơ bản như địa điểm du lịch, số ngày, ngân sách và sở thích cá nhân, hệ thống sẽ tự động đề xuất kế hoạch du lịch phù hợp. Ngoài ra, hệ thống còn hỗ trợ lập kế hoạch thủ công thông qua bản đồ tương tác, cho phép người dùng tự lựa chọn địa điểm và quản lý lịch trình theo nhu cầu riêng.

Việc ứng dụng AI trong lĩnh vực du lịch giúp nâng cao trải nghiệm người dùng, giảm thời gian chuẩn bị và tăng tính cá nhân hóa cho từng chuyến đi. Đây cũng là xu hướng đang được nhiều nền tảng du lịch lớn trên thế giới áp dụng nhằm cải thiện chất lượng dịch vụ và tối ưu khả năng hỗ trợ người dùng.

2.2 Trí tuệ nhân tạo và Prompt Engineering

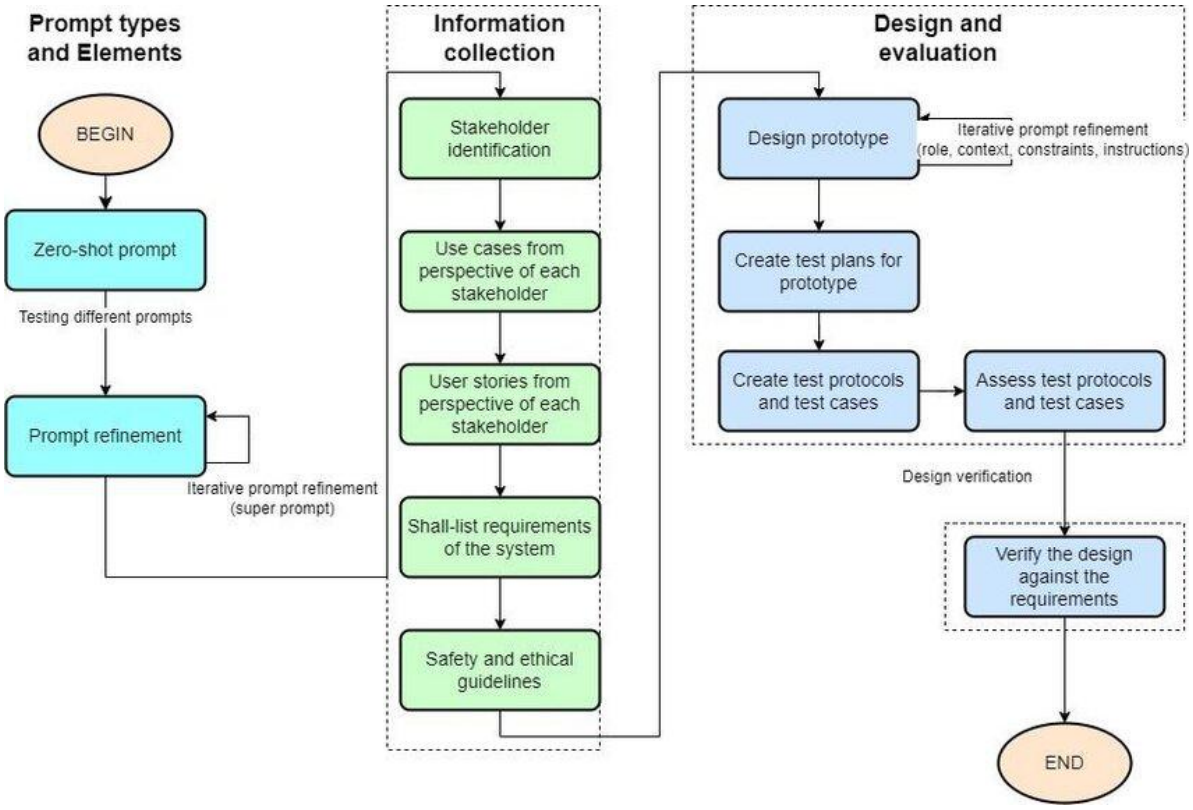
Trí tuệ nhân tạo là lĩnh vực nghiên cứu và phát triển các hệ thống có khả năng mô phỏng tư duy và hành vi của con người thông qua máy tính. AI hiện nay được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, y tế, tài chính, thương mại điện tử và du lịch.

Trong đề tài này, hệ thống sử dụng Google Gemini AI để hỗ trợ tự động sinh lịch trình du lịch. Gemini là mô hình ngôn ngữ lớn do Google phát triển, có khả năng

tiếp nhận yêu cầu bằng ngôn ngữ tự nhiên và sinh nội dung phù hợp dựa trên ngữ cảnh được cung cấp.

Khi người dùng nhập các thông tin như địa điểm du lịch, số ngày, ngân sách và sở thích cá nhân, hệ thống sẽ xây dựng một prompt và gửi đến Gemini AI để tạo ra lịch trình phù hợp.

Prompt Engineering là kỹ thuật thiết kế câu lệnh đầu vào nhằm giúp mô hình AI hiểu rõ yêu cầu và tạo ra kết quả chính xác hơn. Một prompt tốt cần mô tả rõ mục tiêu cần thực hiện, dữ liệu đầu vào, định dạng kết quả mong muốn và các ràng buộc cần tuân thủ.



Hình 2.1: Quy trình Prompt Engineering và tối ưu hóa phản hồi AI

Trong hệ thống, prompt thường bao gồm các yêu cầu như chia lịch trình theo từng ngày, gợi ý địa điểm phù hợp với sở thích, tối ưu thời gian di chuyển và trả kết quả theo định dạng JSON để backend dễ dàng xử lý.

Việc áp dụng Prompt Engineering giúp tăng độ chính xác của kết quả, hạn chế phản hồi sai cấu trúc và nâng cao khả năng cá nhân hóa lịch trình cho từng người dùng.

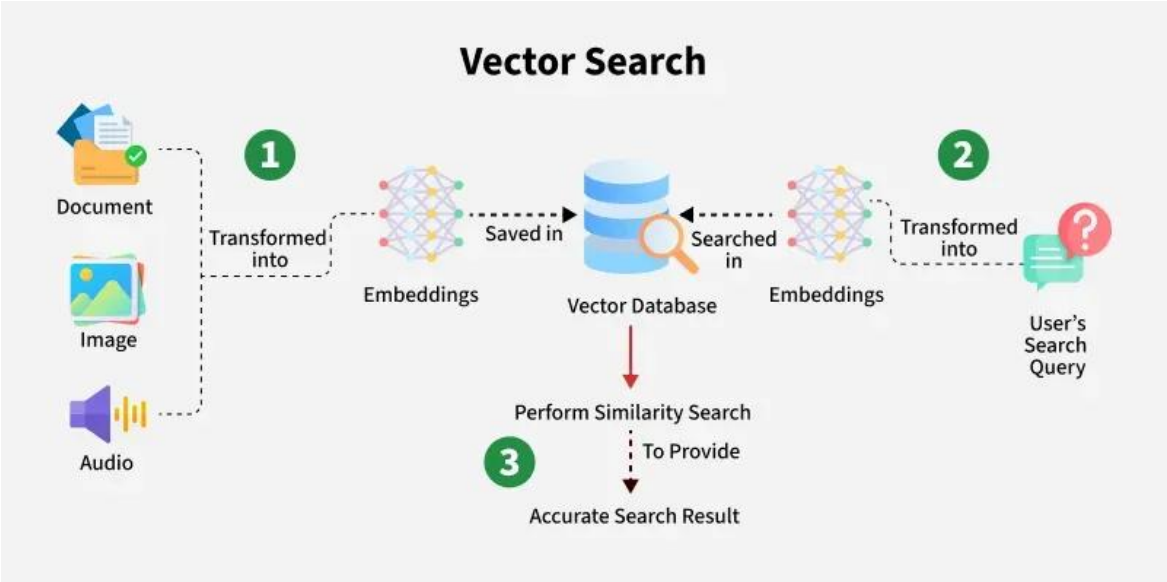
2.3 Vector Database và Semantic Search

Semantic Search là phương pháp tìm kiếm dựa trên ý nghĩa của nội dung thay vì chỉ so khớp từ khóa đơn thuần. Kỹ thuật này cho phép hệ thống hiểu được ngữ cảnh và mục đích của câu truy vấn, từ đó đưa ra kết quả phù hợp hơn với nhu cầu thực tế của người dùng.

Trong hệ thống, Semantic Search được ứng dụng để gợi ý địa điểm du lịch, tìm kiếm theo sở thích và xếp hạng mức độ phù hợp giữa người dùng với địa điểm.

Để thực hiện tìm kiếm ngữ nghĩa, dữ liệu địa điểm được chuyển đổi thành vector embedding thông qua mô hình AI. Mỗi vector biểu diễn ý nghĩa ngữ cảnh của địa điểm dưới dạng dữ liệu số học nhiều chiều.

Vector Database là cơ sở dữ liệu chuyên dùng để lưu trữ và truy vấn các vector embedding. Hệ thống sử dụng kỹ thuật cosine similarity để tính toán mức độ tương đồng giữa câu truy vấn của người dùng và dữ liệu địa điểm trong hệ thống.



Hình 2.2: Quy trình tìm kiếm ngữ nghĩa bằng Vector Embedding

Hình minh họa quá trình chuyển đổi dữ liệu và câu truy vấn thành vector embedding để thực hiện tìm kiếm ngữ nghĩa trong Vector Database. Hệ thống sử dụng phép đo độ tương đồng nhằm tìm ra các kết quả phù hợp nhất với nhu cầu của người dùng.

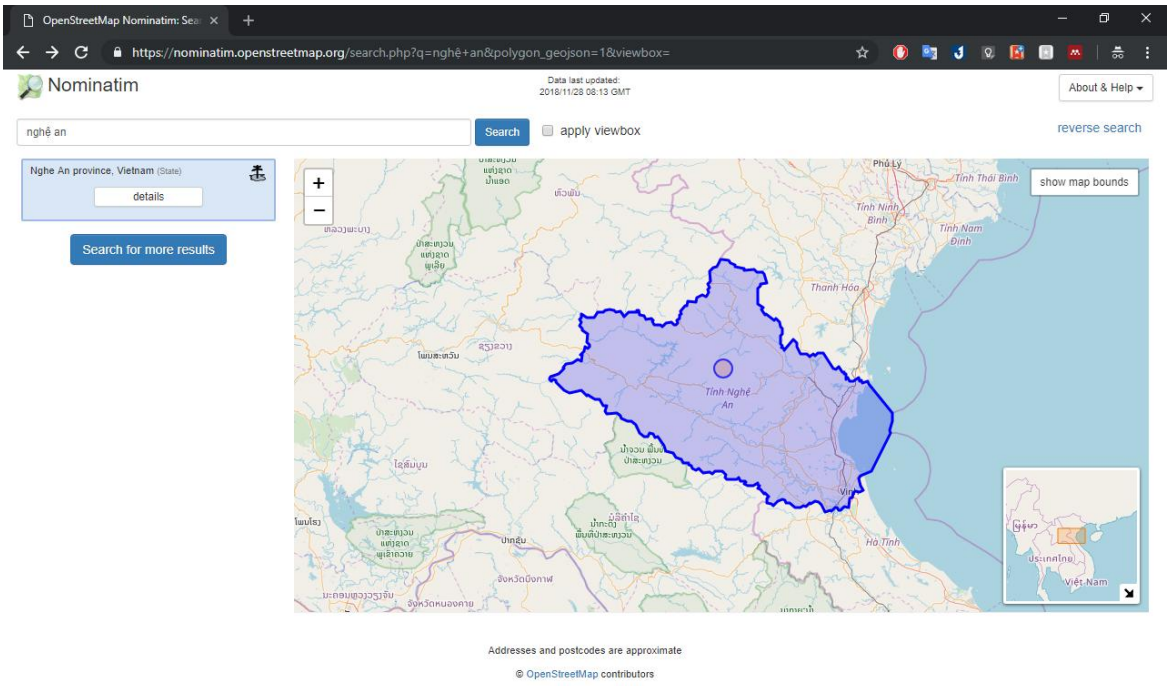
Việc kết hợp Semantic Search và Vector Database giúp tăng độ chính xác tìm kiếm, hiểu ngữ nghĩa người dùng và cải thiện trải nghiệm sử dụng hệ thống.

2.4 OpenStreetMap và Nominatim API

OpenStreetMap là dự án bản đồ mã nguồn mở được xây dựng và cập nhật bởi cộng đồng trên toàn thế giới. Dữ liệu của OpenStreetMap được cung cấp miễn phí và cho phép các nhà phát triển tích hợp vào website hoặc ứng dụng để xây dựng các chức năng liên quan đến bản đồ và vị trí địa lý.

Trong hệ thống, OpenStreetMap được sử dụng để hiển thị bản đồ, tìm kiếm địa điểm, hỗ trợ lập kế hoạch thủ công và đánh dấu vị trí các địa điểm du lịch.

Ngoài ra, hệ thống còn sử dụng Nominatim API để chuyển đổi địa chỉ sang tọa độ, tìm kiếm địa điểm theo tên và hiển thị thông tin vị trí.



Hình 2.3: Giao diện tìm kiếm địa điểm bằng Nominatim API trên nền OpenStreetMap

Một số ưu điểm của OpenStreetMap là miễn phí sử dụng, dữ liệu phong phú, hỗ trợ cộng đồng lớn và dễ tích hợp với các ứng dụng web hiện đại.

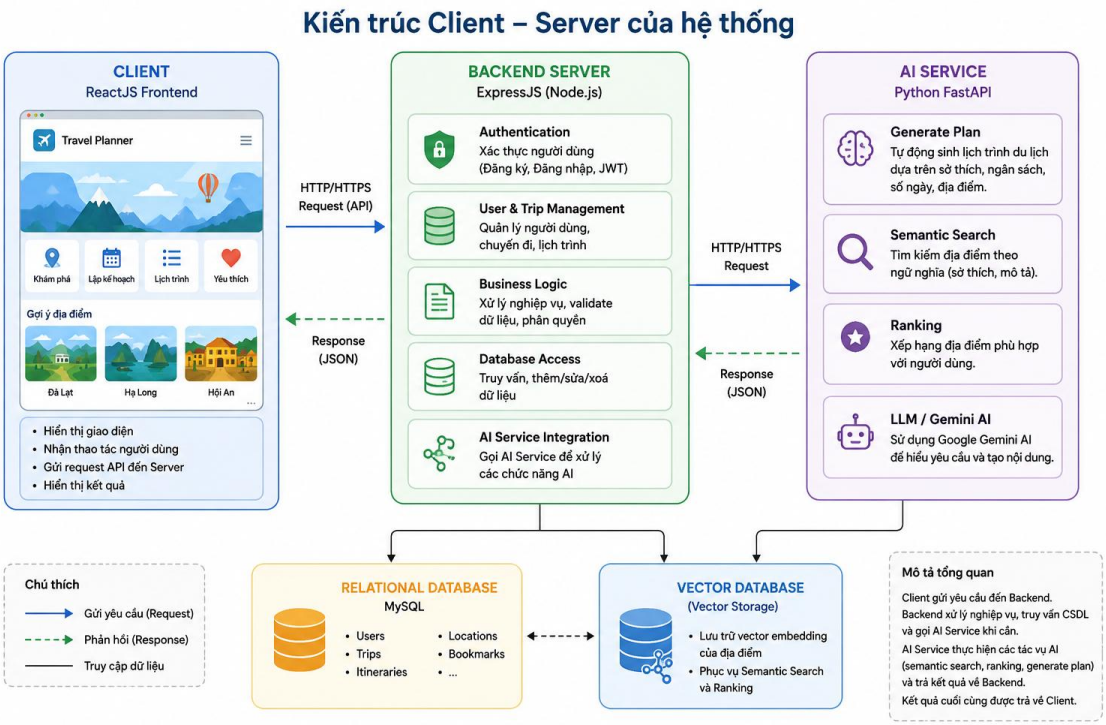
2.5 Kiến trúc Client – Server

Kiến trúc Client – Server là mô hình phổ biến trong phát triển ứng dụng web hiện đại. Trong mô hình này, hệ thống được chia thành hai thành phần chính gồm Client và Server.

Client là thành phần giao tiếp trực tiếp với người dùng, chịu trách nhiệm hiển thị giao diện và gửi request đến server. Trong đề tài, frontend được xây dựng bằng ReactJS.

Server là thành phần xử lý dữ liệu và logic nghiệp vụ. Backend của hệ thống được xây dựng bằng ExpressJS trên nền Node.js, trong khi AI Service được triển khai bằng Python FastAPI.

Frontend đảm nhiệm việc hiển thị giao diện, nhận thao tác người dùng và gửi request API. Backend chịu trách nhiệm xác thực người dùng, xử lý dữ liệu, truy vấn cơ sở dữ liệu và giao tiếp với AI Service. AI Service thực hiện các chức năng liên quan



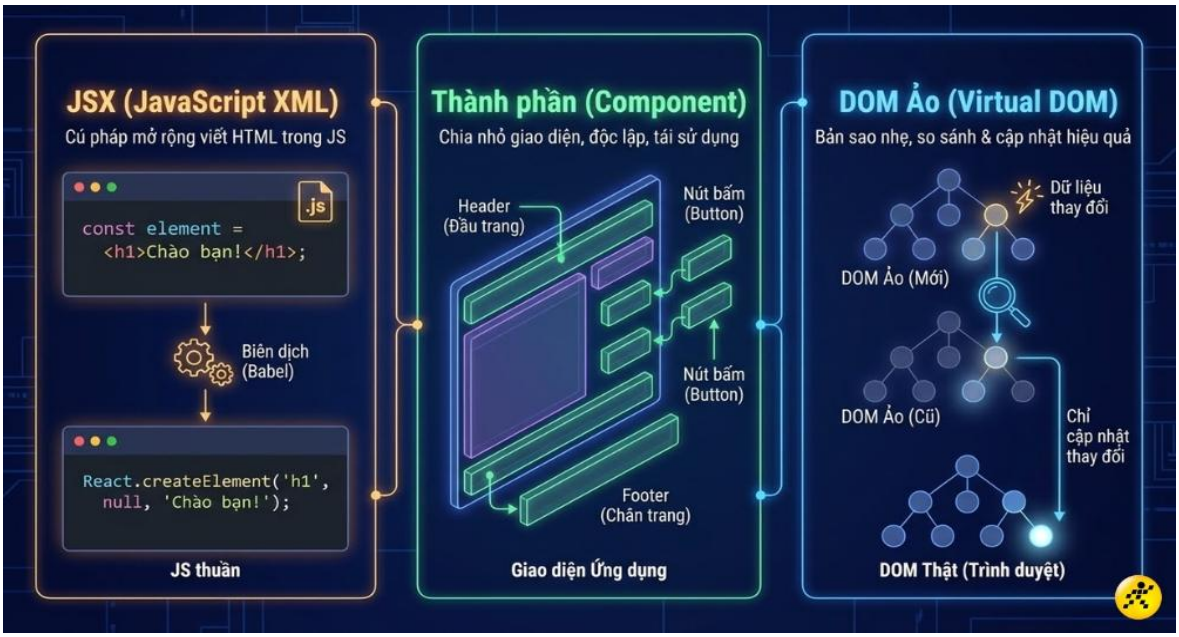
Hình 2.4: Kiến trúc Client – Server của hệ thống

đến trí tuệ nhân tạo như semantic search, ranking và generate plan.

2.6 ReactJS

ReactJS là thư viện JavaScript mã nguồn mở được phát triển bởi Meta nhằm hỗ trợ xây dựng giao diện người dùng cho các ứng dụng web hiện đại. React hoạt động theo hướng component-based, cho phép chia giao diện thành nhiều thành phần nhỏ có thể tái sử dụng.

Một trong những đặc điểm nổi bật của ReactJS là cơ chế Virtual DOM. Khi dữ liệu thay đổi, React chỉ cập nhật các thành phần cần thiết thay vì render lại toàn bộ giao diện, giúp tăng hiệu suất xử lý và cải thiện trải nghiệm người dùng.



Hình 2.5: Các thành phần chính trong ReactJS và cơ chế Virtual DOM

Trong hệ thống, ReactJS được sử dụng để xây dựng các trang như đăng nhập, dashboard, AI Planner, khám phá địa điểm, quản lý chuyến đi và trang quản trị.

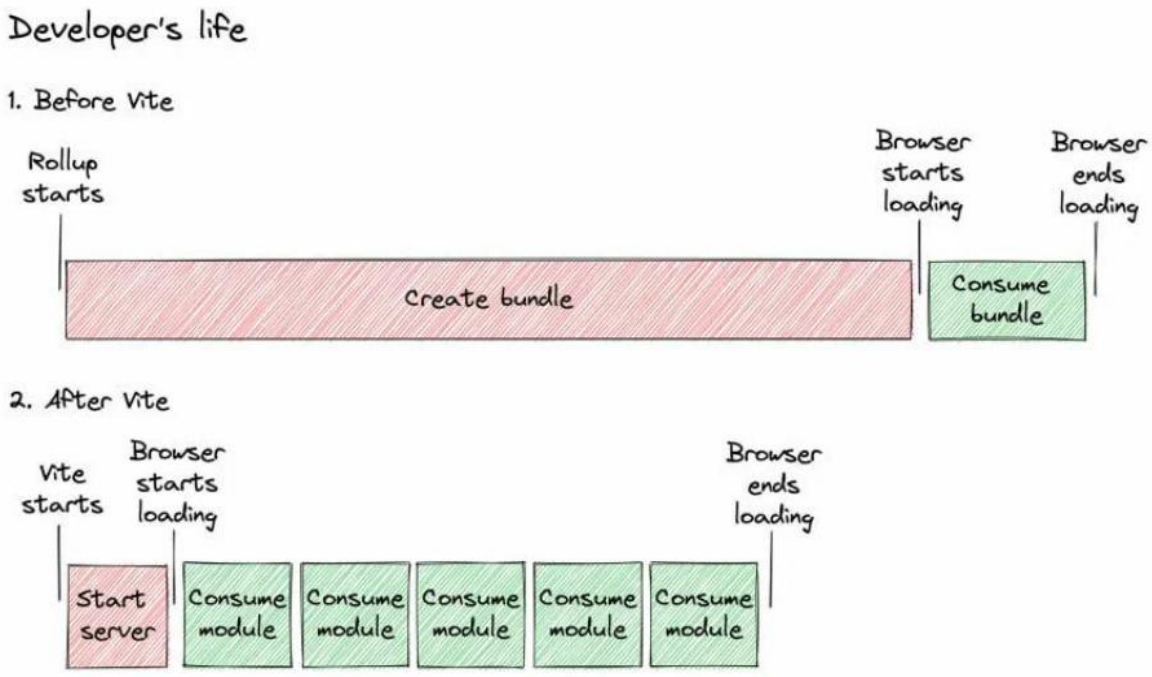
Frontend còn sử dụng React Router để điều hướng giữa các trang, Axios để gọi API và Context API để quản lý trạng thái toàn cục.

Việc sử dụng ReactJS giúp hệ thống có giao diện hiện đại, tốc độ phản hồi nhanh, dễ tái sử dụng component và thuận tiện trong quá trình mở rộng hệ thống.

2.7 Vite

Vite là công cụ build frontend hiện đại được phát triển nhằm tối ưu tốc độ phát triển ứng dụng web. Khác với các công cụ build truyền thống, Vite sử dụng ES Modules và chỉ biên dịch các phần cần thiết khi mã nguồn thay đổi.

Trong đề tài, Vite được sử dụng để khởi tạo và vận hành frontend ReactJS. Một số ưu điểm của Vite là tốc độ khởi động nhanh, hỗ trợ Hot Module Replacement hiệu quả, cấu hình đơn giản và tối ưu tốt cho ReactJS.



Hình 2.6: So sánh tốc độ tải module trước và sau khi sử dụng Vite

Vite giúp giảm đáng kể thời gian reload giao diện trong quá trình phát triển, đặc biệt phù hợp với các ứng dụng có nhiều component và tương tác phức tạp như hệ thống lập kế hoạch du lịch thông minh.

2.8 ExpressJS

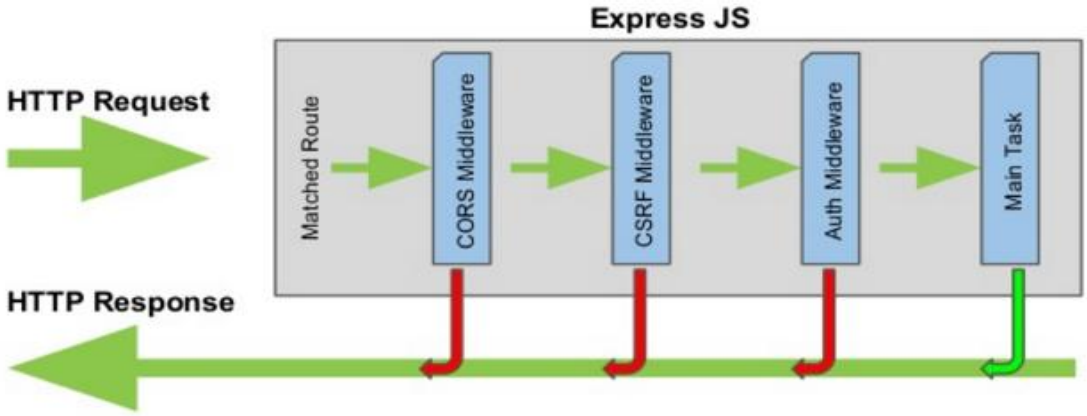
ExpressJS là framework mã nguồn mở chạy trên nền Node.js, được sử dụng phổ biến để xây dựng RESTful API và hệ thống backend cho các ứng dụng web.

Trong hệ thống, ExpressJS đóng vai trò xử lý request từ frontend, xác thực người dùng, quản lý API, giao tiếp với cơ sở dữ liệu và kết nối với AI Service.

Backend được tổ chức thành nhiều thư mục như routes, controllers, services, middlewares và utils nhằm tăng khả năng bảo trì và mở rộng hệ thống.

ExpressJS hỗ trợ middleware giúp kiểm tra dữ liệu đầu vào, xử lý lỗi và xác thực JWT trước khi request được xử lý bởi controller.

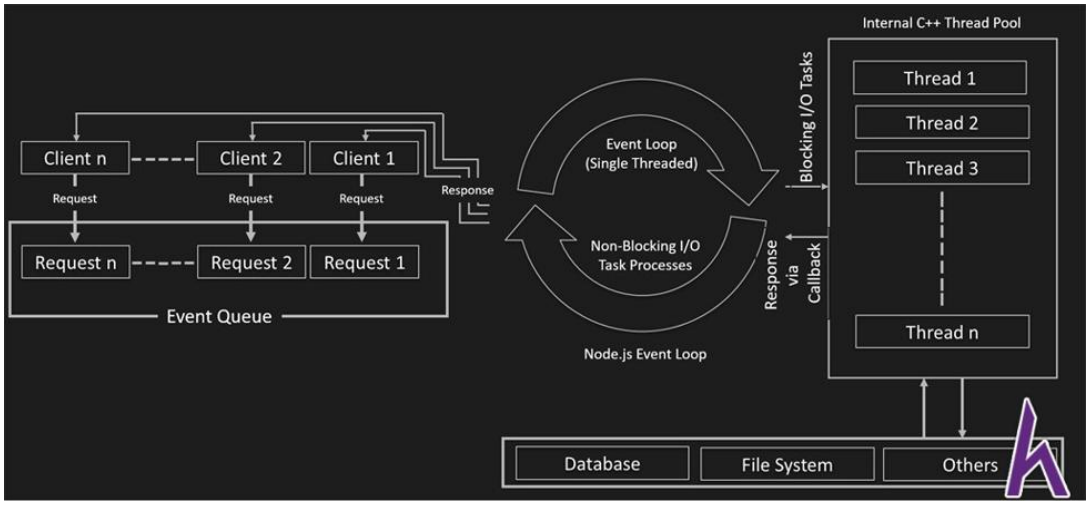
Một số ưu điểm của ExpressJS là cấu trúc đơn giản, khả năng mở rộng tốt, dễ tích hợp với ReactJS và hỗ trợ xây dựng REST API hiệu quả.



Hình 2.7:Kiến trúc xử lý request và response trong ExpressJS

2.9 Node.js

Node.js là môi trường chạy JavaScript phía server được xây dựng trên V8 Engine của Google. Node.js hoạt động theo cơ chế event-driven và non-blocking I/O, cho phép xử lý nhiều request đồng thời với hiệu suất cao.



Hình 2.8: Mô hình Event Loop và Non-blocking I/O trong Node.js

Trong đề tài, Node.js được sử dụng để vận hành backend ExpressJS, quản lý API, xử lý xác thực và giao tiếp với cơ sở dữ liệu.

Một số ưu điểm của Node.js là hiệu suất xử lý tốt, cùng sử dụng JavaScript cho frontend và backend, hệ sinh thái npm phong phú và hỗ trợ mạnh cho các ứng dụng web hiện đại.

2.10 Python FastAPI

FastAPI là framework phát triển API hiện đại của Python, được thiết kế nhằm tối ưu hiệu suất và hỗ trợ xử lý bất đồng bộ.

Trong hệ thống, FastAPI được sử dụng để xây dựng AI Service nhằm xử lý các chức năng như Generate Plan, Hybrid Search, Rank Locations và Organize Plan.

FastAPI hỗ trợ async/await, tự động sinh tài liệu API và kiểm tra kiểu dữ liệu đầu vào thông qua type hinting của Python.

Một số ưu điểm của FastAPI là hiệu suất cao, dễ tích hợp AI và Machine Learning, hỗ trợ Swagger API và thuận tiện cho việc triển khai các mô hình trí tuệ nhân tạo.

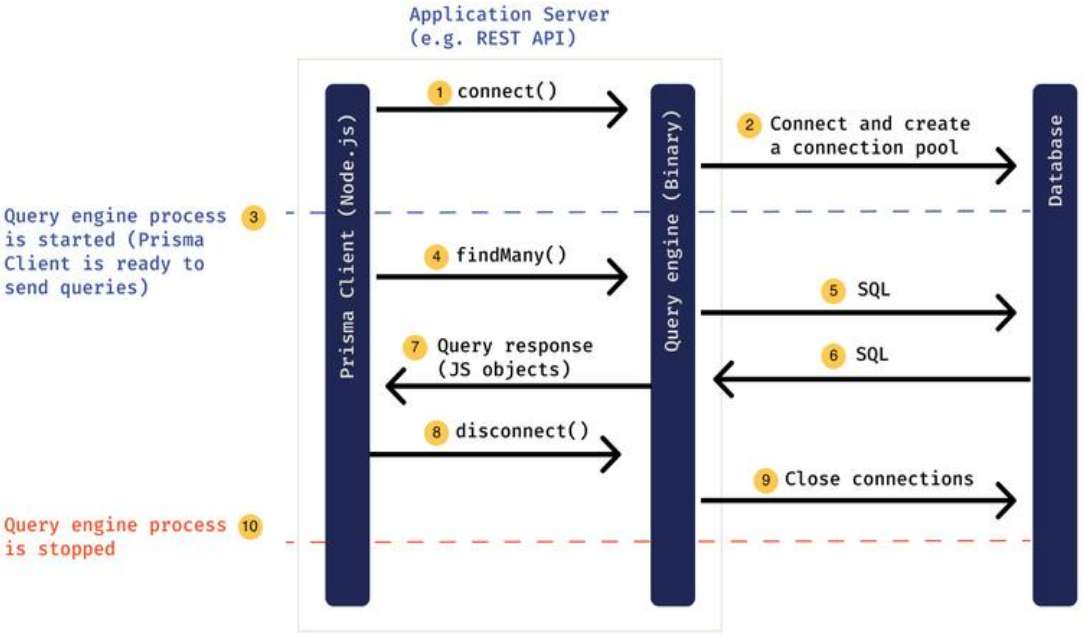
Việc tách AI Service thành một hệ thống riêng giúp backend Node.js giảm tải xử lý và thuận tiện hơn trong việc mở rộng AI sau này.

2.11 Prisma ORM

Prisma ORM là công cụ hỗ trợ thao tác cơ sở dữ liệu trong môi trường Node.js và TypeScript. Prisma đóng vai trò trung gian giữa backend và database, giúp lập trình viên thao tác dữ liệu bằng code thay vì viết trực tiếp SQL.

Trong hệ thống, Prisma ORM được sử dụng để kết nối MySQL, quản lý schema dữ liệu và thực hiện các thao tác CRUD.

Tệp schema.prisma được dùng để định nghĩa các model dữ liệu và mối quan hệ giữa các bảng. Từ schema này, Prisma sẽ tự động sinh Prisma Client để backend truy vấn dữ liệu.



Hình 2.9: Quy trình truy vấn cơ sở dữ liệu bằng Prisma ORM

Một số ưu điểm của Prisma ORM là hỗ trợ type-safe query, dễ quản lý schema, hỗ trợ migration và giảm lỗi khi thao tác với cơ sở dữ liệu.

Việc sử dụng Prisma giúp mã nguồn backend rõ ràng hơn, hạn chế lỗi SQL và tăng tốc độ phát triển hệ thống.

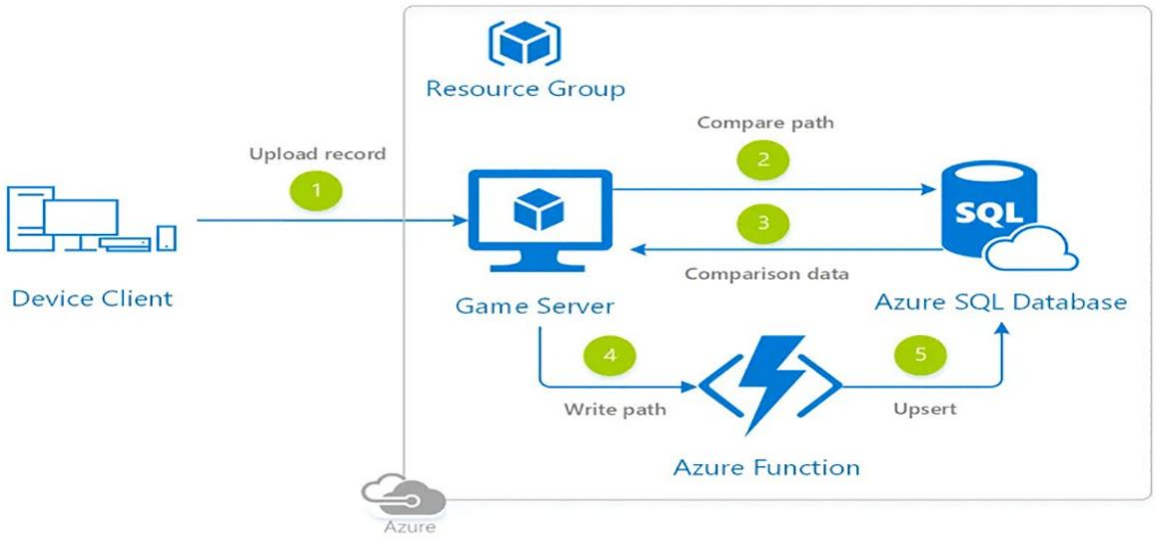
2.12 MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ được sử dụng phổ biến trong các ứng dụng web hiện nay. MySQL hỗ trợ lưu trữ dữ liệu dưới dạng bảng và quản lý mối quan hệ giữa các bảng thông qua khóa chính và khóa ngoại

Trong hệ thống, MySQL được sử dụng để lưu trữ thông tin người dùng, địa điểm, lịch trình, đánh giá và dữ liệu sở thích cá nhân.

Một số ưu điểm của MySQL là hiệu suất xử lý tốt, hỗ trợ dữ liệu lớn, cộng đồng sử dụng rộng rãi và dễ triển khai.

MySQL được kết nối với backend thông qua Prisma ORM nhằm tăng độ an toàn khi truy vấn và đồng bộ schema với mã nguồn backend.

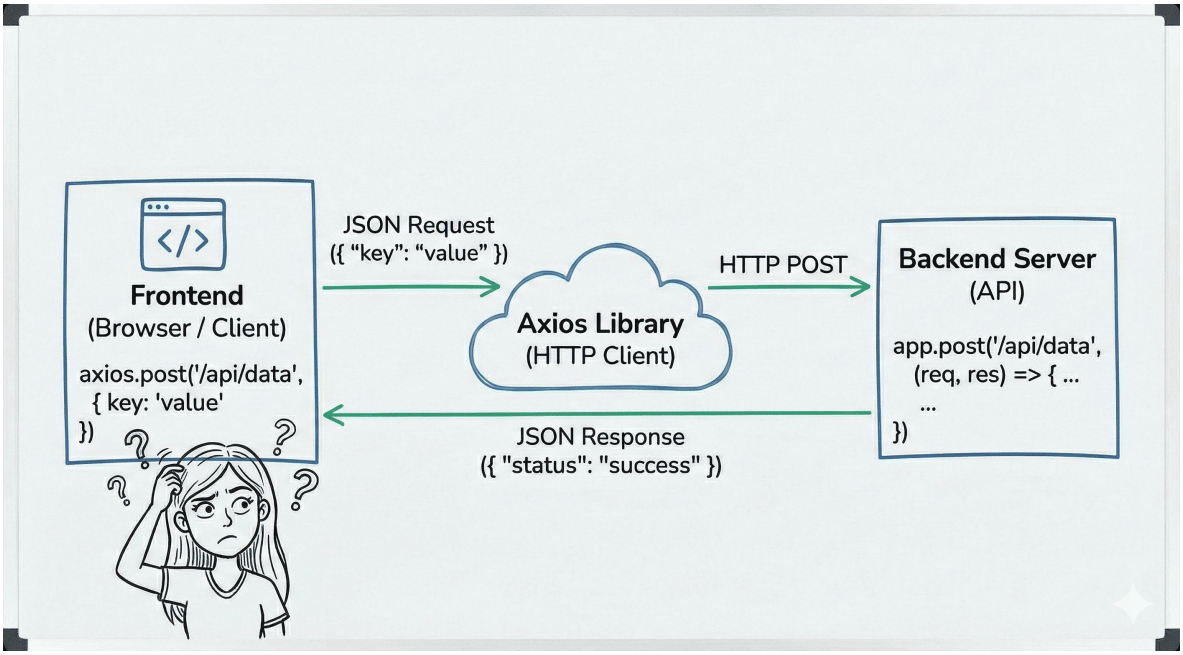


Hình 2.10: Mô hình kết nối và xử lý dữ liệu với MySQL Database

2.13 Axios

Axios là thư viện JavaScript được sử dụng để gửi HTTP request giữa frontend và backend. Axios hoạt động dựa trên Promise và hỗ trợ các phương thức như GET, POST, PUT và DELETE.

Trong hệ thống, Axios được sử dụng để gửi request đăng nhập, tạo lịch trình, lấy dữ liệu địa điểm và quản lý chuyến đi.



Hình 2.11: Mô hình giao tiếp API giữa Frontend và Backend bằng Axios

Một số ưu điểm của Axios là cú pháp đơn giản, hỗ trợ async/await, tự động chuyển đổi JSON, hỗ trợ interceptor và dễ xử lý lỗi request.

2.14 LeafletJS

LeafletJS là thư viện JavaScript mã nguồn mở chuyên dùng để xây dựng các ứng dụng bản đồ tương tác trên nền web.

Trong hệ thống, LeafletJS được sử dụng kết hợp với OpenStreetMap nhằm hiển thị bản đồ, đánh dấu địa điểm, tìm kiếm vị trí và hỗ trợ lập kế hoạch thủ công.



Hình 2.12: LeafletJS tích hợp bản đồ tương tác trong ứng dụng web

Hệ thống sử dụng React Leaflet để tích hợp LeafletJS trực tiếp vào React component. Điều này giúp frontend dễ quản lý dữ liệu bản đồ và thuận tiện trong việc render dữ liệu động.

Một số ưu điểm của LeafletJS là nhẹ, tối ưu hiệu năng, dễ tích hợp ReactJS, miễn phí sử dụng và hỗ trợ nhiều plugin mở rộng.

Người dùng có thể tìm kiếm địa điểm, chọn vị trí trên bản đồ và thêm điểm đến trực tiếp vào lịch trình du lịch của mình.

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM

3.1 Phân tích yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

Hệ thống cần đáp ứng các chức năng chính như quản lý tài khoản, tạo lịch trình bằng AI, lập kế hoạch thủ công, quản lý chuyến đi và khám phá địa điểm.

Đối với người dùng, hệ thống hỗ trợ đăng ký tài khoản, đăng nhập, cập nhật thông tin cá nhân, tạo lịch trình tự động bằng AI và quản lý các chuyến đi đã lưu.

Ngoài ra, người dùng còn có thể sử dụng chức năng lập kế hoạch thủ công thông qua bản đồ tương tác để lựa chọn địa điểm và xây dựng lịch trình theo nhu cầu riêng.

Hệ thống cũng hỗ trợ tìm kiếm địa điểm, xem đánh giá, xem thông tin du lịch và gợi ý địa điểm phù hợp với sở thích của người dùng.

Đối với quản trị viên, hệ thống hỗ trợ quản lý người dùng, quản lý dữ liệu địa điểm, quản lý đánh giá và kiểm duyệt nội dung hệ thống.

3.1.2 Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống còn cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo hiệu suất và trải nghiệm sử dụng.

Về hiệu năng, hệ thống cần phản hồi nhanh, hỗ trợ nhiều request đồng thời và đảm bảo API hoạt động ổn định.

Về khả năng mở rộng, kiến trúc hệ thống được chia thành frontend, backend và AI Service riêng biệt nhằm thuận tiện cho việc nâng cấp và phát triển thêm tính năng trong tương lai.

3.1.3 Các tác nhân hệ thống

Hệ thống bao gồm hai tác nhân chính là User và Admin.

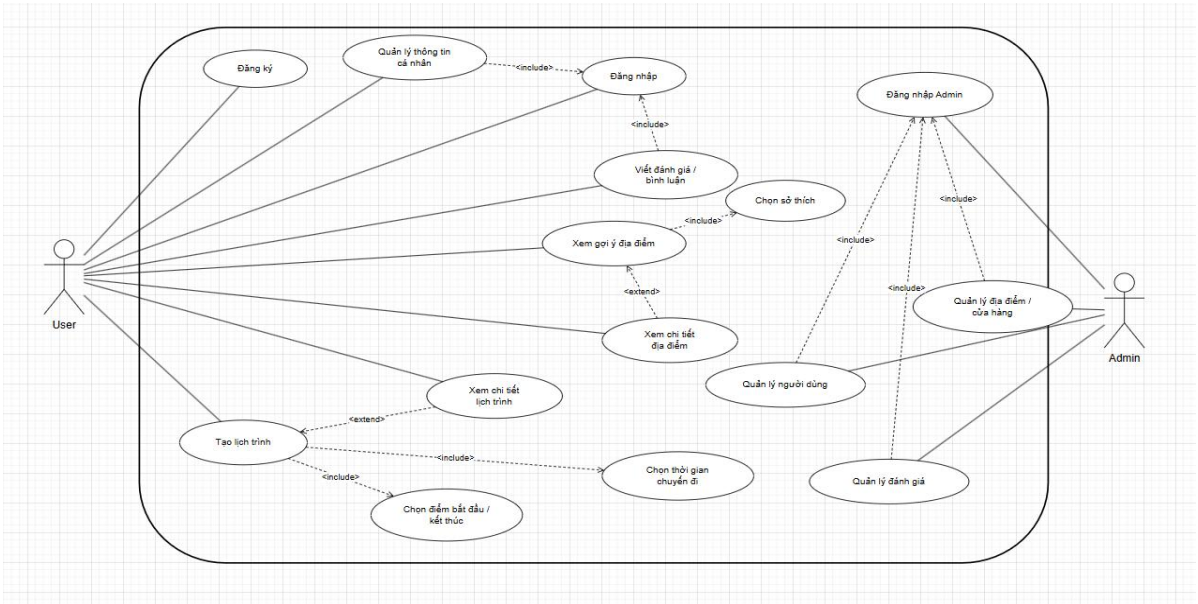
User là người sử dụng trực tiếp hệ thống để lập kế hoạch du lịch, tìm kiếm địa điểm, quản lý chuyến đi và đánh giá địa điểm.

Admin là người quản trị hệ thống, chịu trách nhiệm quản lý dữ liệu người dùng, quản lý địa điểm và kiểm duyệt nội dung nhằm đảm bảo hệ thống hoạt động ổn định.

3.2 Phân tích và thiết kế hệ thống

3.2.1 Sơ đồ Use Case tổng quát

Use Case Diagram được sử dụng nhằm mô tả các chức năng chính của hệ thống và mối quan hệ giữa các tác nhân với hệ thống.



Hình 3.1: USE CASE DIAGRAM hệ thống

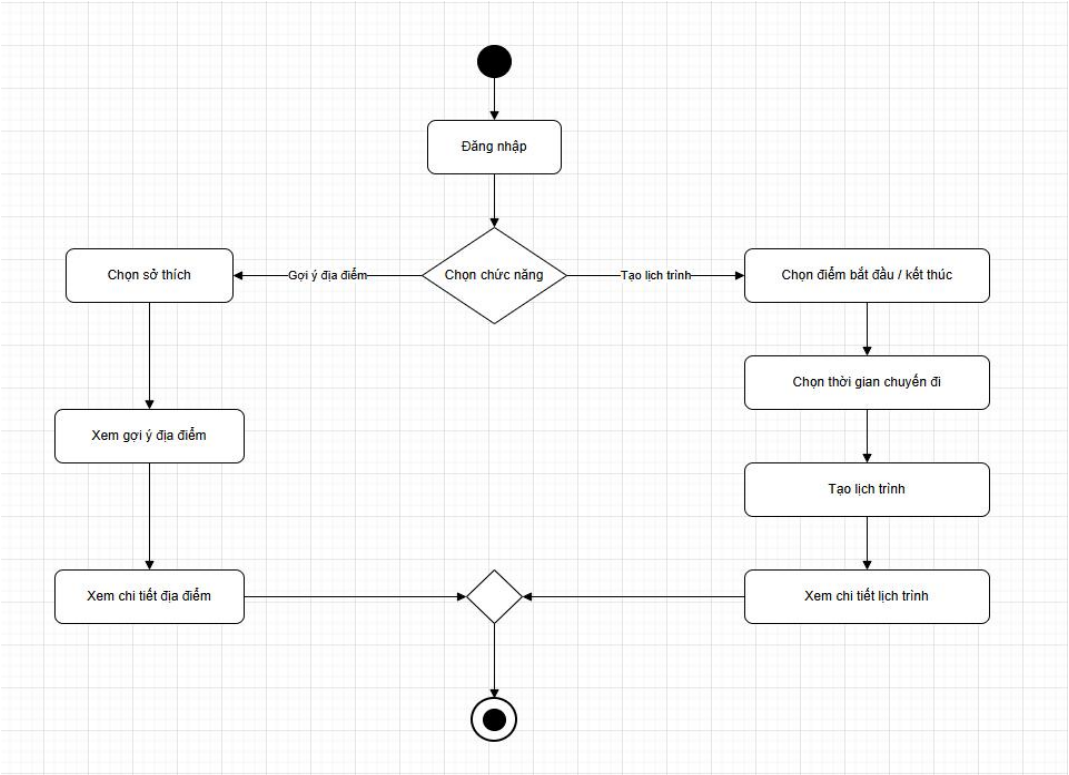
Sơ đồ thể hiện hai tác nhân chính là User và Admin. User có thể thực hiện các chức năng như đăng ký, đăng nhập, lập kế hoạch bằng AI, quản lý chuyến đi và đánh giá địa điểm. Admin có quyền quản lý người dùng, quản lý dữ liệu và kiểm duyệt nội dung hệ thống.

Các chức năng chính của User gồm đăng ký tài khoản, đăng nhập, tạo lịch trình bằng AI, lập kế hoạch thủ công, khám phá địa điểm và quản lý chuyến đi.

Đối với Admin, hệ thống hỗ trợ quản lý người dùng, quản lý địa điểm và quản lý đánh giá nhằm đảm bảo dữ liệu hoạt động ổn định.

3.2.2 Sơ đồ Activity Diagram

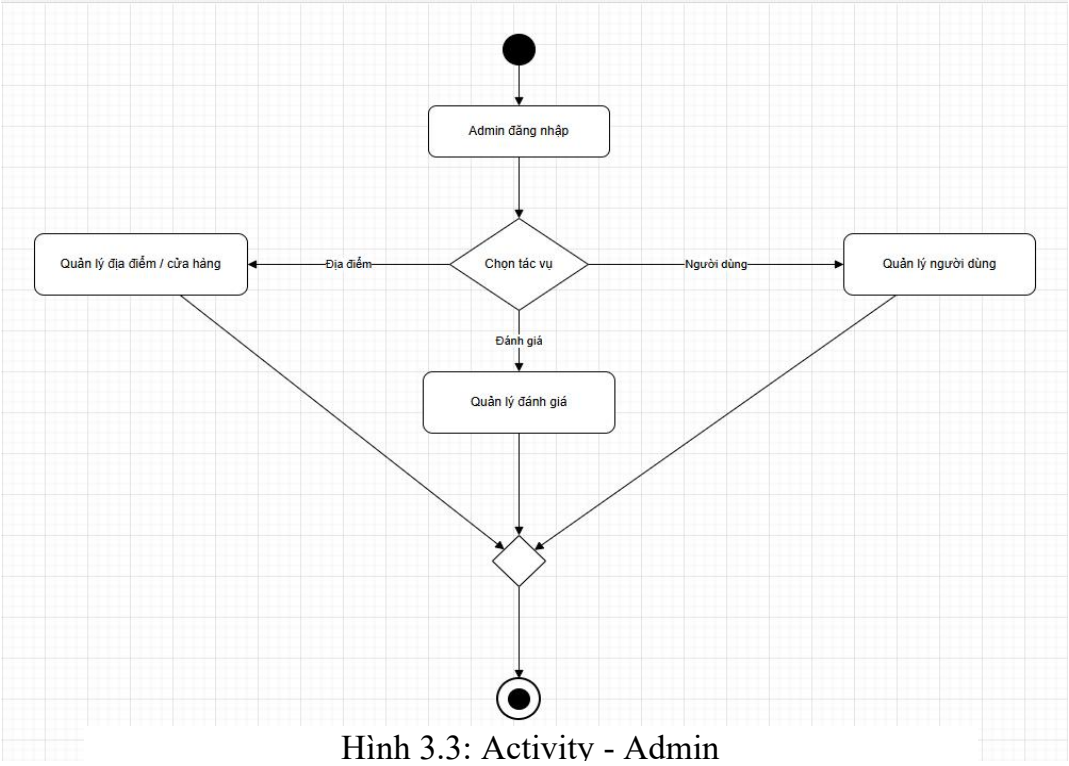
Activity Diagram được sử dụng nhằm mô tả luồng xử lý của hệ thống theo từng chức năng cụ thể.



Hình 3.2: Activity - User

Sơ đồ mô tả luồng xử lý của người dùng từ lúc đăng nhập cho đến khi sử dụng các chức năng như AI Planner, Manual Planner và quản lý chuyến đi.

Quy trình hoạt động bao gồm đăng nhập hệ thống, lựa chọn chức năng, nhập dữ liệu chuyến đi, gửi request đến backend và nhận kết quả trả về từ hệ thống.



Hình 3.3: Activity - Admin

Sơ đồ mô tả quá trình quản trị viên đăng nhập và thực hiện các chức năng quản lý dữ liệu trong hệ thống.

Admin có thể quản lý người dùng, quản lý địa điểm và kiểm duyệt đánh giá nhằm đảm bảo hệ thống hoạt động ổn định.

3.2.3 Thiết kế cơ sở dữ liệu (ERD)

Hệ thống sử dụng MySQL để lưu trữ dữ liệu và Prisma ORM để quản lý truy vấn dữ liệu.

Sơ đồ ERD mô tả các bảng dữ liệu chính và mối quan hệ giữa chúng như users, itineraries, locations, reviews và itinerary_items.

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ nhằm đảm bảo khả năng lưu trữ ổn định, giảm trùng lặp dữ liệu và thuận tiện cho việc truy vấn.

Các bảng chính của hệ thống gồm users, locations, itineraries, itinerary_items, reviews và user_preferences.

Hệ thống sử dụng khóa ngoại để liên kết dữ liệu giữa các bảng nhằm đảm bảo tính toàn vẹn dữ liệu và tăng hiệu quả truy vấn.

3.2.4 Thiết kế kiến trúc hệ thống

Hệ thống được xây dựng theo mô hình Client – Server kết hợp với AI Service độc lập nhằm đảm bảo khả năng mở rộng, dễ bảo trì và tối ưu hiệu suất xử lý. Kiến trúc tổng thể của hệ thống bao gồm frontend ReactJS, backend ExpressJS, AI Service FastAPI, MySQL Database, Gemini AI API và OpenStreetMap API.

Sơ đồ mô tả kiến trúc tổng thể của hệ thống lập kế hoạch du lịch thông minh. Frontend ReactJS giao tiếp với backend ExpressJS thông qua REST API. Backend xử lý nghiệp vụ, kết nối MySQL bằng Prisma ORM và giao tiếp với AI Service FastAPI để xử lý các chức năng liên quan đến trí tuệ nhân tạo.

Frontend đóng vai trò giao tiếp trực tiếp với người dùng, chịu trách nhiệm hiển thị giao diện và gửi request API. Backend đảm nhiệm xử lý nghiệp vụ, xác thực người dùng, quản lý dữ liệu và điều phối các request liên quan đến AI. Trong khi đó, AI Service được triển khai độc lập bằng Python FastAPI nhằm xử lý các chức năng như semantic search, hybrid search, rank locations và generate plan.

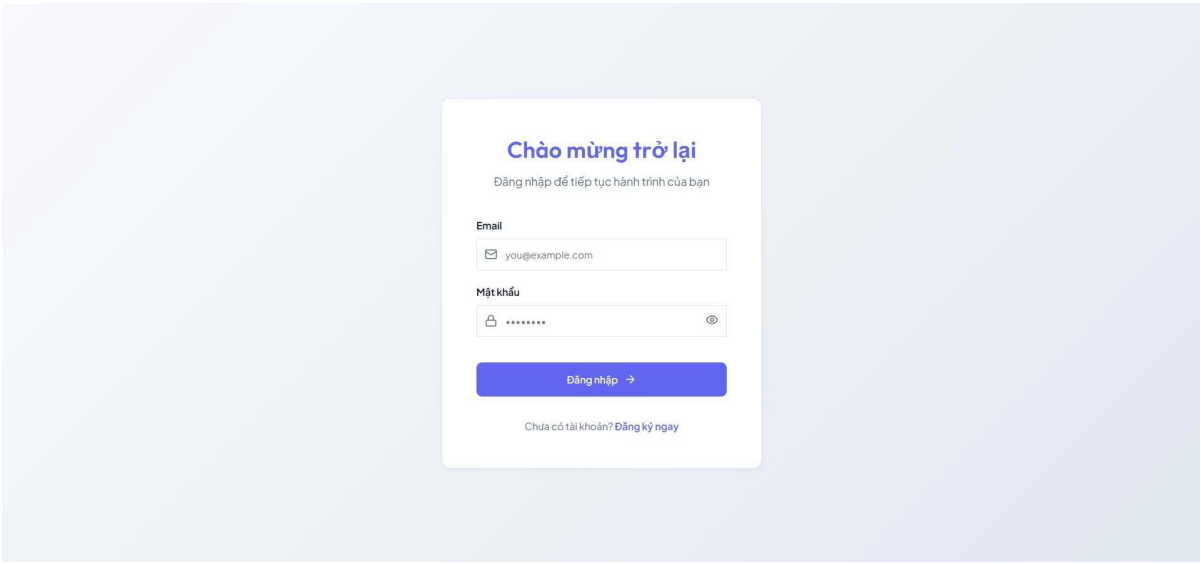
Kiến trúc hiện tại giúp hệ thống dễ mở rộng, giảm tải backend chính, thuận tiện bảo trì và tăng khả năng tích hợp AI trong tương lai.

3.3 Xây dựng và triển khai hệ thống

Sau khi hoàn thành giai đoạn phân tích và thiết kế, nhóm tiến hành xây dựng hệ thống theo kiến trúc đã đề ra. Quá trình phát triển được chia thành ba phần chính gồm frontend, backend và AI Service. Ngoài ra, hệ thống còn được triển khai kết nối với MySQL Database, Gemini AI API và OpenStreetMap API nhằm phục vụ các chức năng liên quan đến dữ liệu và trí tuệ nhân tạo.

3.3.1 Xây dựng frontend

Frontend của hệ thống được phát triển bằng ReactJS kết hợp với Vite và Tailwind CSS. Giao diện được xây dựng theo hướng component-based nhằm tăng khả năng tái sử dụng và thuận tiện cho việc bảo trì.



Hình 3.4: Giao diện đăng nhập hệ thống

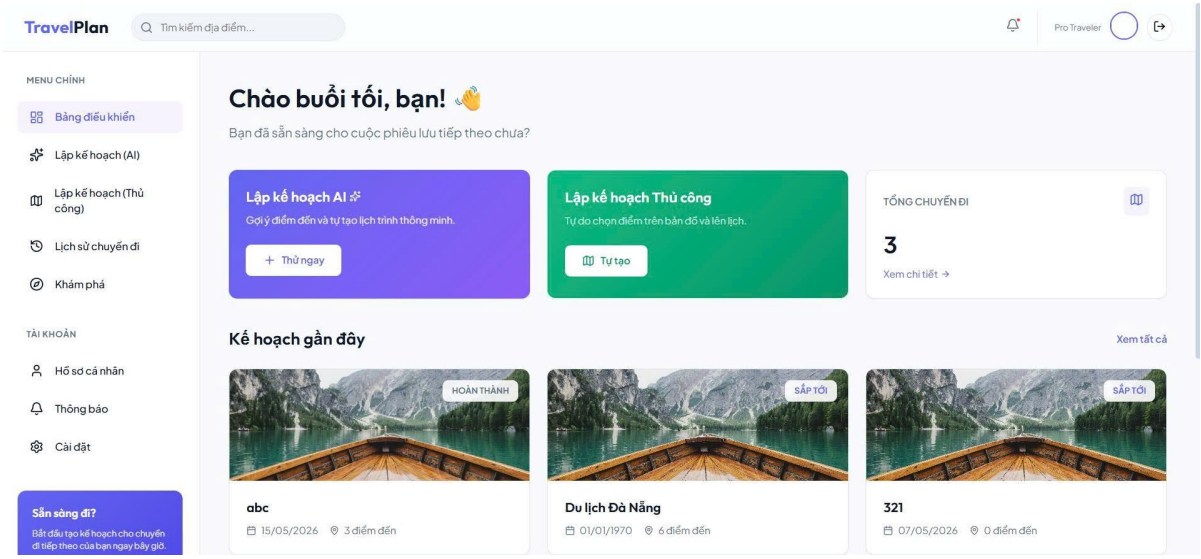
Giao diện đăng nhập được thiết kế đơn giản, tập trung vào trải nghiệm người dùng với biểu mẫu nhập email và mật khẩu, hỗ trợ hiển thị và ẩn mật khẩu, đồng thời cung cấp liên kết chuyển sang trang đăng ký nếu người dùng chưa có tài khoản. Sau khi xác thực thành công, người dùng được chuyển đến bảng điều khiển chính của hệ thống.

Frontend được tổ chức thành nhiều thư mục như pages, components, services, hooks, contexts và utils. Trong đó, pages chứa các giao diện chính của hệ thống,

components chứa các thành phần giao diện dùng chung, services quản lý API và contexts hỗ trợ quản lý trạng thái toàn cục.

React Router được sử dụng để điều hướng giữa các trang, Axios được dùng để gửi request API và Context API hỗ trợ quản lý trạng thái người dùng trong toàn bộ ứng dụng.

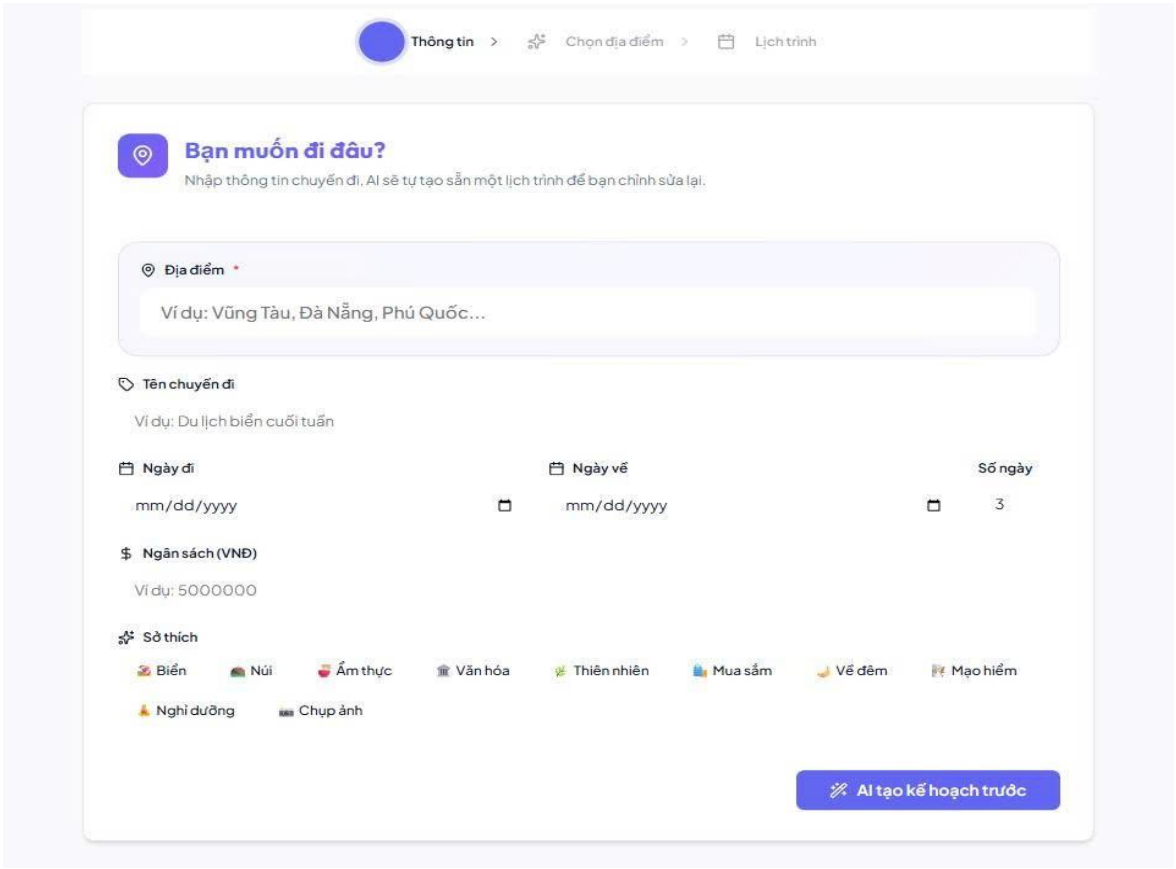
Dashboard đóng vai trò trung tâm điều hướng chính của hệ thống, cho phép người dùng truy cập nhanh đến các chức năng như AI Planner, Manual Planner, Explore Locations và quản lý chuyến đi.



Hình 3.5: Giao diện bảng điều khiển chính (Dashboard)

Dashboard hiển thị hai chức năng quan trọng là lập kế hoạch bằng AI và lập kế hoạch thủ công. Ngoài ra, hệ thống còn hiển thị thống kê số lượng chuyến đi, danh sách các kế hoạch gần đây và menu quản lý tài khoản như hồ sơ cá nhân, thông báo và cài đặt.

AI Planner là chức năng nổi bật nhất của hệ thống, hỗ trợ người dùng tự động tạo lịch trình bằng trí tuệ nhân tạo.

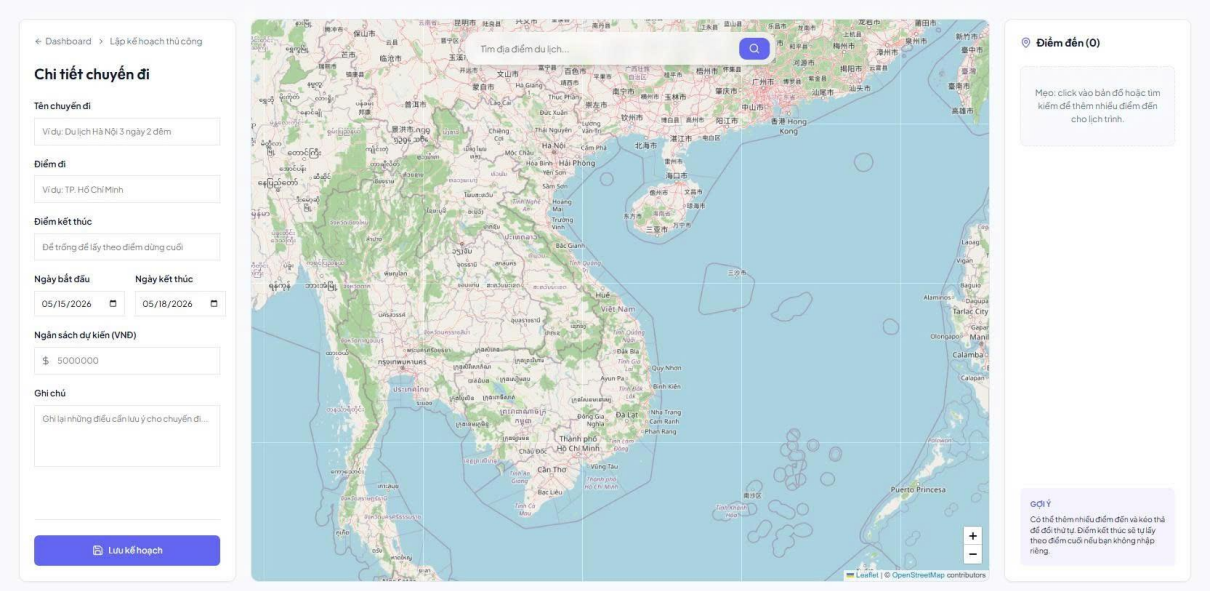


Hình 3.6: Giao diện nhập thông tin cho AI Planner

Người dùng nhập các thông tin như địa điểm mong muốn, tên chuyến đi, ngày khởi hành, ngày kết thúc, ngân sách dự kiến và các sở thích du lịch như biển, núi, ẩm thực, văn hóa hoặc nghỉ dưỡng. Sau khi gửi yêu cầu, frontend hiển thị trạng thái xử lý và nhận dữ liệu từ backend để trình bày lịch trình được AI đề xuất theo từng ngày.

Khi người dùng gửi yêu cầu, frontend sẽ gửi request đến backend thông qua Axios và hiển thị kết quả lịch trình được AI sinh ra.

Ngoài AI Planner, hệ thống còn hỗ trợ lập kế hoạch thủ công thông qua bản đồ tương tác.

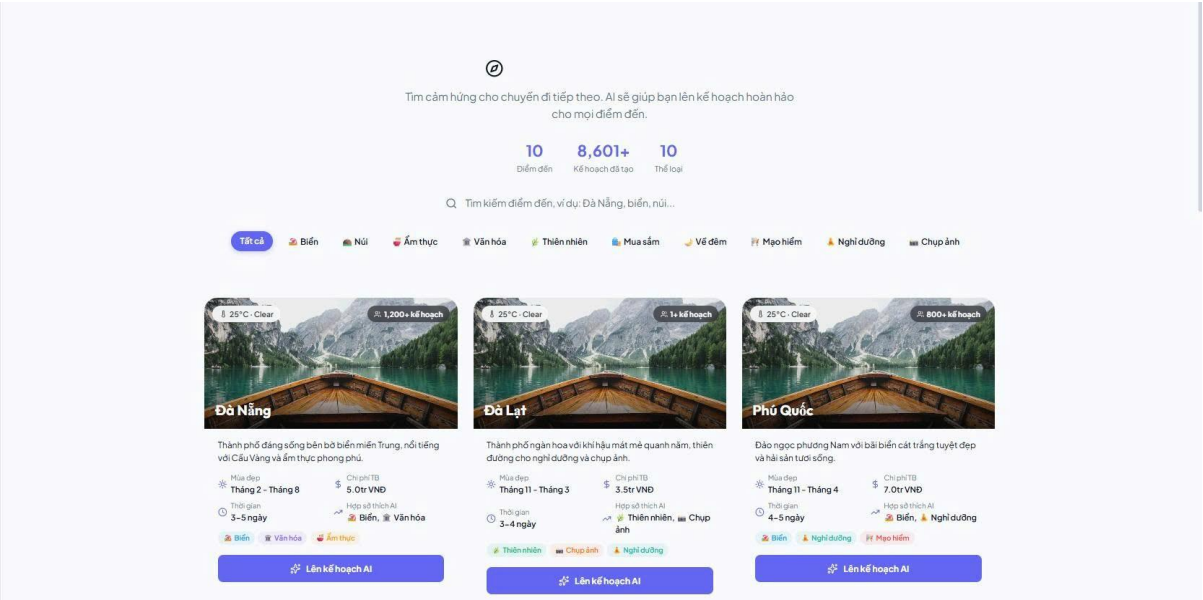


Hình 3.7: Giao diện lập kế hoạch thủ công với bản đồ tương tác

Giao diện này tích hợp bản đồ OpenStreetMap, cho phép người dùng tìm kiếm và lựa chọn địa điểm trực tiếp trên bản đồ. Các điểm đến được thêm vào danh sách bên phải, đồng thời người dùng có thể nhập thông tin về thời gian, ngân sách và ghi chú cho chuyến đi trước khi lưu kế hoạch.

LeafletJS và OpenStreetMap được sử dụng nhằm hỗ trợ hiển thị bản đồ, marker địa điểm và tìm kiếm vị trí trực quan.

Ngoài ra, hệ thống còn hỗ trợ trang khám phá địa điểm nhằm giúp người dùng tìm kiếm các địa điểm du lịch phù hợp với sở thích cá nhân.



Hình 3.8: Giao diện khám phá và gợi ý địa điểm du lịch

Trang Explore Locations cung cấp danh sách các điểm đến nổi bật được hệ thống gợi ý dựa trên dữ liệu lưu trữ và sở thích của người dùng. Mỗi thẻ địa điểm hiển thị hình ảnh, mô tả ngắn, thời gian du lịch phù hợp, chi phí trung bình và các nhóm sở thích tương ứng. Từ đây, người dùng có thể lựa chọn địa điểm và chuyển trực tiếp sang chức năng lập kế hoạch bằng AI.

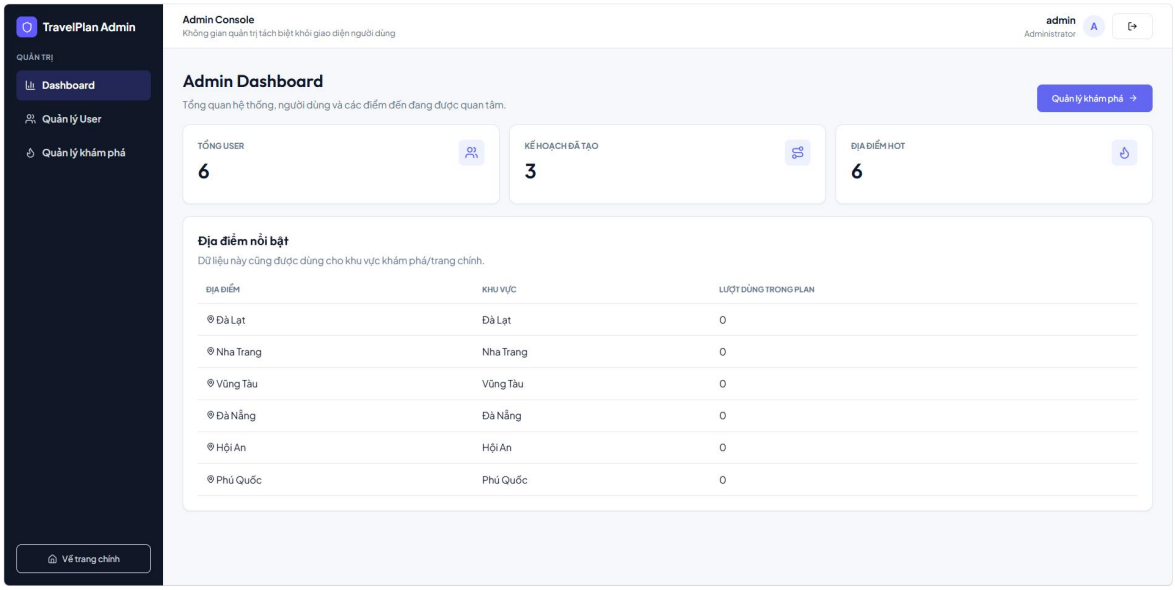
Frontend còn hỗ trợ responsive nhằm đảm bảo giao diện hiển thị tốt trên desktop, tablet và mobile. Một số ưu điểm của ReactJS và Tailwind CSS là tốc độ phát triển giao diện nhanh, dễ tái sử dụng component, hỗ trợ quản lý state hiệu quả và thuận tiện cho việc mở rộng hệ thống.

3.3.2 Xây dựng backend

Backend của hệ thống được phát triển bằng ExpressJS trên nền Node.js nhằm đảm nhiệm vai trò xử lý nghiệp vụ, xác thực người dùng và quản lý dữ liệu. Hệ thống backend được xây dựng theo mô hình RESTful API, cho phép frontend và backend hoạt động độc lập nhưng vẫn đảm bảo khả năng trao đổi dữ liệu hiệu quả.

Source backend được tổ chức thành nhiều thư mục như routes, controllers, services, middlewares và prisma nhằm tăng khả năng bảo trì và mở rộng hệ thống. Prisma ORM được sử dụng để kết nối MySQL và hỗ trợ thực hiện các thao tác CRUD dữ liệu. Một số ưu điểm của Prisma ORM là hỗ trợ type-safe query, dễ quản lý schema, hỗ trợ migration và giảm lỗi khi thao tác với cơ sở dữ liệu.

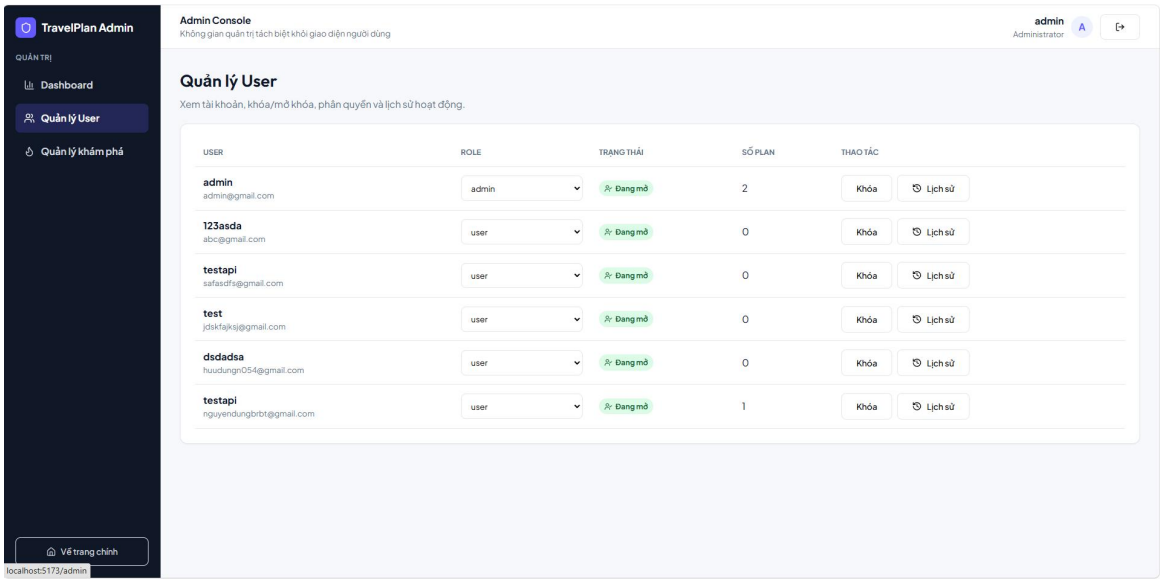
Ngoài chức năng xử lý API cho người dùng, backend còn xây dựng hệ thống quản trị dành riêng cho Admin nhằm hỗ trợ quản lý dữ liệu hệ thống. Các chức năng quản trị bao gồm: quản lý người dùng, phân quyền tài khoản, khóa/mở khóa tài khoản, quản lý địa điểm khám phá, thống kê dữ liệu hệ thống.



Hình 3.9: Giao diện Admin Dashboard

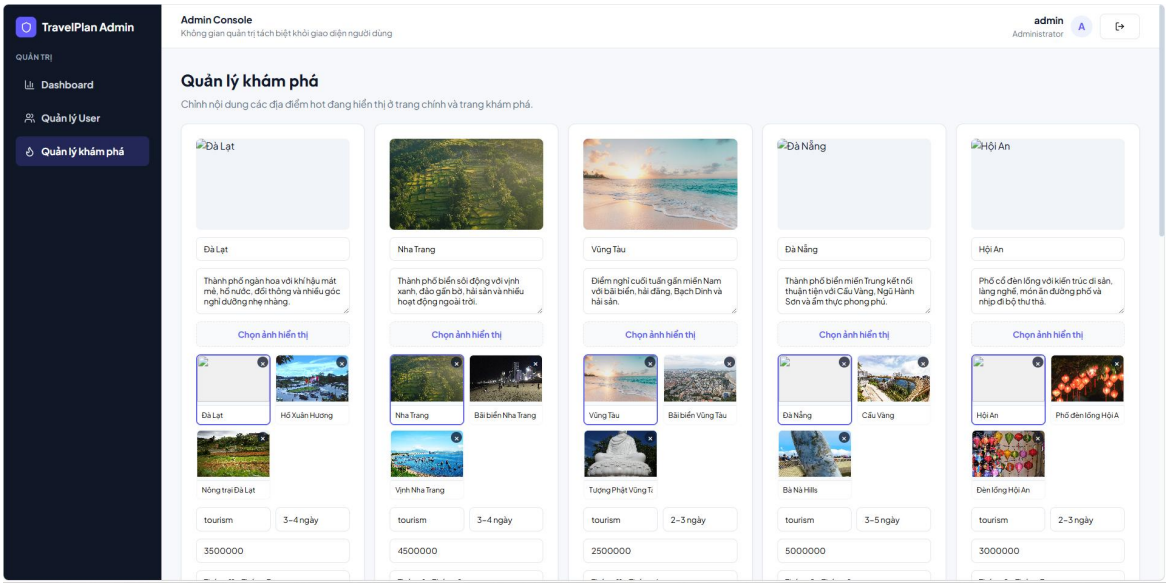
Admin Dashboard hiển thị tổng quan dữ liệu hệ thống như tổng số người dùng, số kế hoạch đã tạo và các địa điểm nổi bật đang được sử dụng trong hệ thống.

Backend cung cấp API để lấy dữ liệu thống kê và quản lý nội dung hiển thị trên dashboard. Các dữ liệu này được truy vấn từ MySQL thông qua Prisma ORM và trả về frontend dưới dạng JSON.



Hình 3.10: Giao diện quản lý người dùng

Trang quản lý người dùng cho phép Admin xem danh sách tài khoản, phân quyền vai trò, khóa/mở khóa tài khoản và theo dõi số lượng kế hoạch của từng người dùng.



Hình 3.11:Giao diện quản lý khám phá địa điểm

Backend xử lý các chức năng xác thực quyền Admin trước khi cho phép thực hiện các thao tác quản trị nhằm đảm bảo tính bảo mật của hệ thống.

Admin có thể quản lý nội dung địa điểm nổi bật, chỉnh sửa mô tả, thay đổi hình ảnh và cập nhật dữ liệu hiển thị tại trang khám phá của hệ thống.

Các dữ liệu địa điểm được backend lưu trữ và quản lý thông qua Prisma ORM kết hợp MySQL nhằm đảm bảo tính ổn định và đồng bộ dữ liệu giữa frontend và backend.

Việc sử dụng ExpressJS và Node.js giúp backend có khả năng xử lý tốt, dễ mở rộng và thuận tiện trong quá trình phát triển hệ thống.

3.3.3 Tích hợp trí tuệ nhân tạo

Trí tuệ nhân tạo là thành phần quan trọng nhất của hệ thống, giúp tự động hóa quá trình lập kế hoạch du lịch và nâng cao khả năng cá nhân hóa trải nghiệm người dùng.

Hệ thống sử dụng Google Gemini AI kết hợp với Prompt Engineering và Semantic Search nhằm tạo ra các lịch trình phù hợp với nhu cầu thực tế của từng người dùng.

AI Service được triển khai độc lập bằng Python FastAPI nhằm tăng khả năng mở rộng, giảm tải backend Node.js, dễ tích hợp AI model và thuận tiện bảo trì trong tương lai.

Khi người dùng sử dụng chức năng AI Planner, frontend sẽ gửi dữ liệu chuyến đi đến backend. Backend tiến hành xây dựng prompt và gửi request đến AI Service để xử lý.

AI Service sử dụng Gemini AI để phân tích dữ liệu đầu vào, tìm kiếm địa điểm phù hợp, xếp hạng địa điểm, tổ chức lịch trình và sinh kế hoạch du lịch hoàn chỉnh.

Prompt Engineering được sử dụng nhằm giúp AI hiểu rõ yêu cầu của người dùng và trả kết quả theo định dạng ổn định. Prompt thường bao gồm địa điểm du lịch, số ngày, ngân sách, sở thích cá nhân và yêu cầu định dạng dữ liệu JSON.

Ngoài ra, hệ thống còn áp dụng Hybrid Search nhằm kết hợp giữa keyword search và semantic search để nâng cao độ chính xác khi tìm kiếm địa điểm du lịch.

Sau khi tìm kiếm địa điểm, AI tiếp tục đánh giá và xếp hạng mức độ phù hợp dựa trên sở thích cá nhân, ngân sách và loại hình chuyến đi trước khi tiến hành xây dựng lịch trình.

Việc tích hợp AI giúp hệ thống giảm thời gian lập kế hoạch, tăng khả năng cá nhân hóa và nâng cao trải nghiệm người dùng.

3.3.4 Triển khai hệ thống

Hệ thống được triển khai theo mô hình tách biệt giữa frontend, backend, AI Service và database nhằm tăng khả năng mở rộng và dễ bảo trì.

Frontend ReactJS được chạy thông qua Vite Development Server để hỗ trợ quá trình phát triển và kiểm thử giao diện. Backend ExpressJS hoạt động như một REST API Server tiếp nhận request từ frontend và xử lý dữ liệu nghiệp vụ. Trong khi đó, AI Service FastAPI được triển khai riêng để xử lý các chức năng liên quan đến AI như semantic search và generate plan.

Database MySQL được sử dụng để lưu trữ dữ liệu người dùng, lịch trình và địa điểm du lịch. Prisma Migration được sử dụng để tạo và đồng bộ cấu trúc cơ sở dữ liệu từ tệp schema.prisma.

Các công nghệ và công cụ được sử dụng trong quá trình triển khai gồm ReactJS, ExpressJS, FastAPI, MySQL, Prisma ORM, Postman, Visual Studio Code và GitHub.

Kết quả triển khai cho thấy frontend hiển thị ổn định, backend xử lý API chính xác, AI Service hoạt động đúng chức năng và dữ liệu được lưu trữ ổn định trong

MySQL. Các chức năng như đăng nhập, AI Planner, Manual Planner, tìm kiếm địa điểm và quản lý lịch trình đều hoạt động đúng theo yêu cầu đề ra.

CHƯƠNG 4. KẾT LUẬN VÀ KIẾN NGHỊ

4.1 Kết luận

Sau quá trình nghiên cứu và phát triển, đề tài "Hệ thống lập kế hoạch du lịch thông minh ứng dụng trí tuệ nhân tạo" đã hoàn thành các chức năng chính như đăng nhập, lập kế hoạch bằng AI, lập kế hoạch thủ công, tìm kiếm địa điểm và quản lý chuyến đi.

Hệ thống được xây dựng với frontend ReactJS, backend ExpressJS, AI Service FastAPI và cơ sở dữ liệu MySQL. Ngoài ra, hệ thống còn tích hợp Gemini AI và OpenStreetMap nhằm hỗ trợ tạo lịch trình và hiển thị bản đồ tương tác.

Kết quả kiểm thử cho thấy các chức năng hoạt động tương đối ổn định, dữ liệu được xử lý chính xác và AI có khả năng sinh lịch trình phù hợp với yêu cầu của người dùng.

Tuy nhiên, hệ thống vẫn còn một số hạn chế như dữ liệu địa điểm chưa phong phú, AI đôi lúc chưa tối ưu lịch trình hoàn toàn và chưa hỗ trợ nhiều tính năng nâng cao.

Nhìn chung, đề tài đã đáp ứng tốt các mục tiêu đặt ra và có khả năng tiếp tục phát triển trong tương lai.

4.2 Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế trong quá trình phát triển và triển khai thực tế.

Dữ liệu địa điểm hiện tại chủ yếu được lấy từ nguồn miễn phí nên chưa thực sự phong phú và đồng bộ. Một số địa điểm còn thiếu hình ảnh, mô tả hoặc thông tin đánh giá chi tiết.

Ngoài ra, AI tuy đã có khả năng sinh lịch trình tự động nhưng trong một số trường hợp kết quả vẫn chưa tối ưu hoàn toàn về thời gian di chuyển và phân bổ hoạt động.

Hệ thống hiện mới được triển khai ở mức đồ án học tập nên chưa tối ưu hiệu năng cho số lượng lớn người dùng đồng thời. Một số chức năng nâng cao như realtime

collaboration, recommendation system hay đồng bộ đa nền tảng vẫn chưa được triển khai.

Bên cạnh đó, hệ thống hiện chủ yếu hoạt động trên nền tảng web và chưa có ứng dụng mobile riêng cho người dùng.

4.3 Hướng phát triển

Trong tương lai, hệ thống có thể được cải thiện và mở rộng thêm nhiều chức năng nhằm nâng cao trải nghiệm người dùng và tăng khả năng ứng dụng thực tế.

Một số hướng phát triển gồm cải thiện AI Planner để tạo lịch trình chính xác hơn, bổ sung thêm dữ liệu địa điểm và tích hợp thêm các API du lịch thực tế như Google Places hoặc Geoapify.

Bên cạnh đó, nhóm cũng có thể phát triển ứng dụng mobile nhằm hỗ trợ người dùng thuận tiện hơn trong quá trình sử dụng.

Trong tương lai, hệ thống cũng có thể được tối ưu hiệu năng và triển khai trên cloud server nhằm hỗ trợ nhiều người dùng đồng thời và tăng khả năng mở rộng hệ thống.

4.4 Kiến nghị

Trong tương lai, nhóm mong muốn tiếp tục cải thiện và phát triển hệ thống nhằm nâng cao khả năng ứng dụng thực tế.

Hệ thống có thể được bổ sung thêm dữ liệu du lịch từ các nền tảng lớn nhằm tăng độ chính xác và phong phú cho chức năng tìm kiếm địa điểm.

Ngoài ra, cần tiếp tục tối ưu AI Planner để lịch trình được cá nhân hóa tốt hơn và phù hợp hơn với nhu cầu thực tế của người dùng.

Nhóm cũng kiến nghị phát triển thêm các tính năng như collaborative planning, recommendation system và ứng dụng mobile nhằm nâng cao trải nghiệm sử dụng.

Bên cạnh đó, hệ thống có thể được triển khai trên cloud server và tối ưu hiệu năng nhằm hỗ trợ nhiều người dùng đồng thời trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

[1] https://link.springer.com/article/10.1007/s10115-021-01648-3

[2] https://www.sciencedirect.com/science/article/pii/S1568494624001042

[3] https://www.thegioididong.com/hoi-dap/react-la-gi-tim-hieu-ve-reactjs-1587503

[4] https://expressjs.com/

[5]https://fastapi.tiangolo.com/

[6] https://fptshop.com.vn/tin-tuc/for-gamers/prisma-la-gi-197834

[7] https://cellphones.com.vn/sforum/mysql-la-gi

[8] https://ai.google.dev/

[9] https://dothanhlong.org/free-osm-geocoding-services/

[10] https://leafletjs.com/

[11] https://axios-http.com/docs/intro

[12]https://www.researchgate.net/figure/The-prompt-engineering-design-process_fig1_384160181

[13] https://www.geeksforgeeks.org/nlp/semantic-vs-vector-search/

[14] https://200lab.io/blog/vite-la-gi

[15] https://howkteam.vn/course/so-tay-javascript/nodejs-hoat-dong-nhu-the-nao-4083